



ECOLE
POLYTECHNIQUE
DE BRUXELLES

01. Introduction to computer graphics

INFO-H502

Daniele Bonatto

LISA-VR

2024

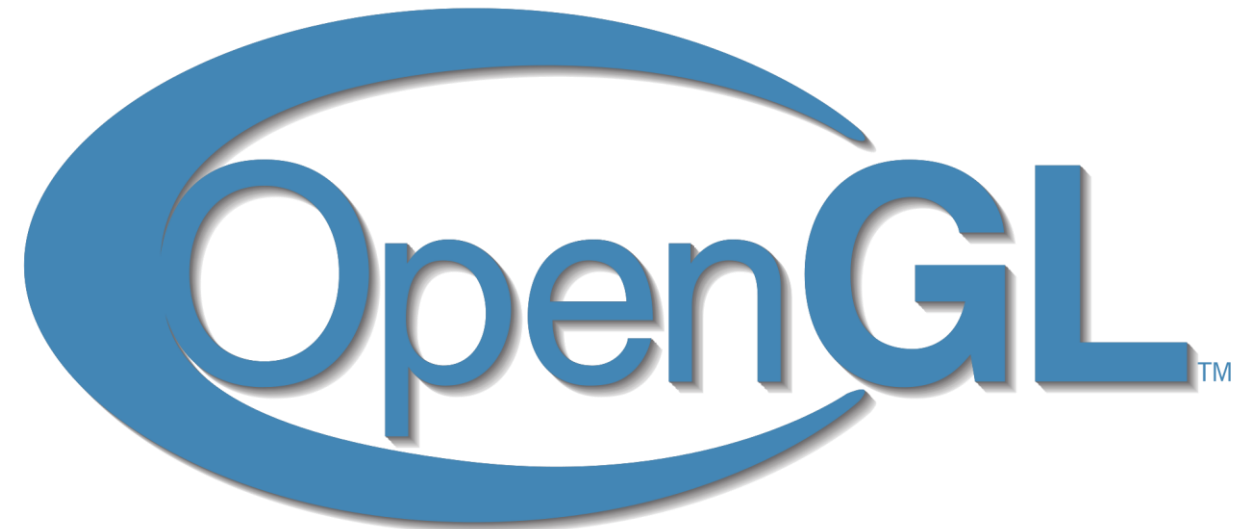
Disclaimer and Ressources

- Several images and shaders in the slides
 - Shamelessly token and adapted from all around the web (CMU, MIT, Books, etc.)

- Ressources:

- OpenGL

- <http://www.learnopengl.com/>
 - <http://ogldev.atSPACE.co.uk/>
 - <https://www.opengl.org/registry/>



- Books:

- Mathematics for 3D game programming and computer graphics
 - Real-Time Rendering
 - Physically based rendering
 - Computer Graphics Principles



What is computer graphics

- Goal: Displaying 3D images on the screen
 - 3D (classical rendering)
 - Realistic (Raytracing)
 - Real-time
- Needed:
 - All the mathematics (the lectures)
 - Programming (Exercises)
- What do we mean by computer graphics ?











GOD OF WAR
RAGNARÖK



LVL 1





THE
DAUNTLESS

Meet with the Caravan Traders

SUNNY

ZERONTA



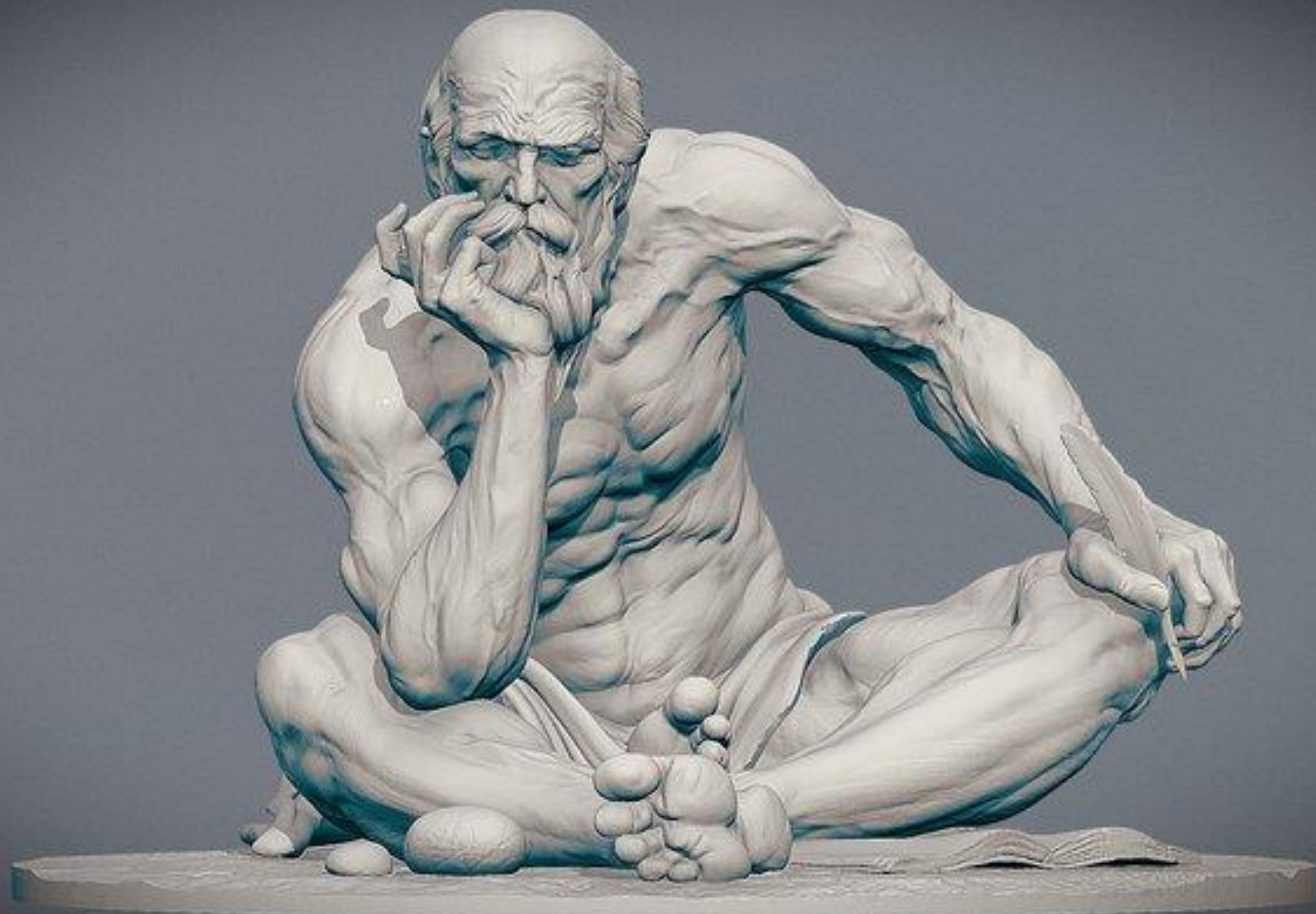
87 / 207



U







What does a computer contains ?

Screen



Tower



Input devices

What does a computer contains ?



Main memory

What does a computer contains ?



Processor

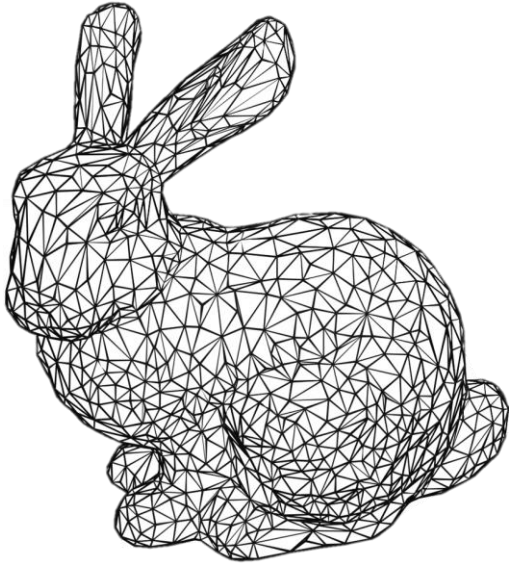
What does a computer contains ?



Graphical Processing Unit (GPU)

What is computer graphics ?

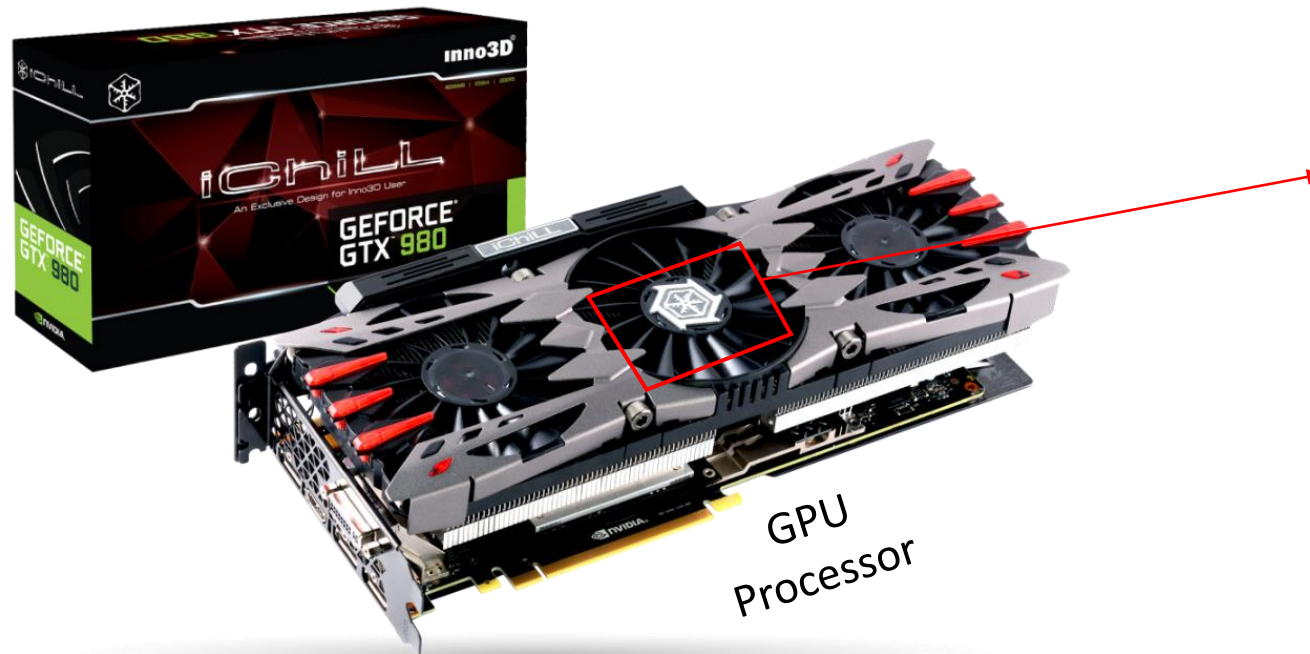
- Goal: Displaying 3D objects on the screen
 - 3D (classical rendering)
 - Realistic (Raytracing)



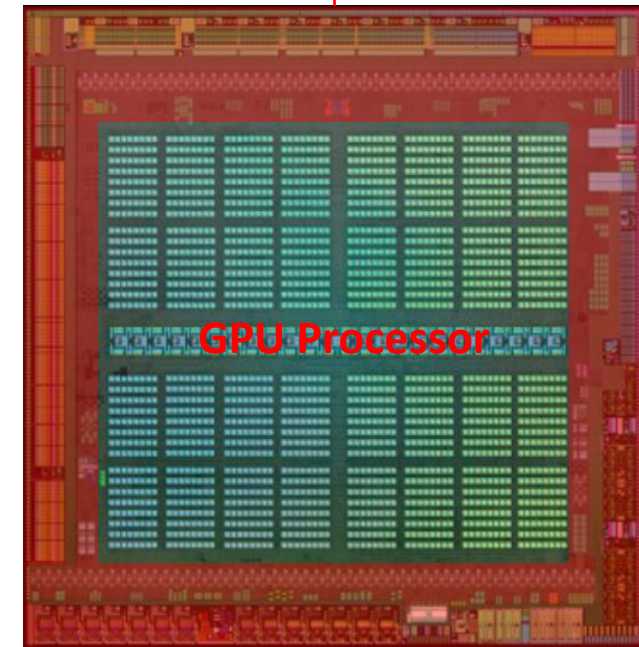
What is a GPU ?



What is a GPU ?

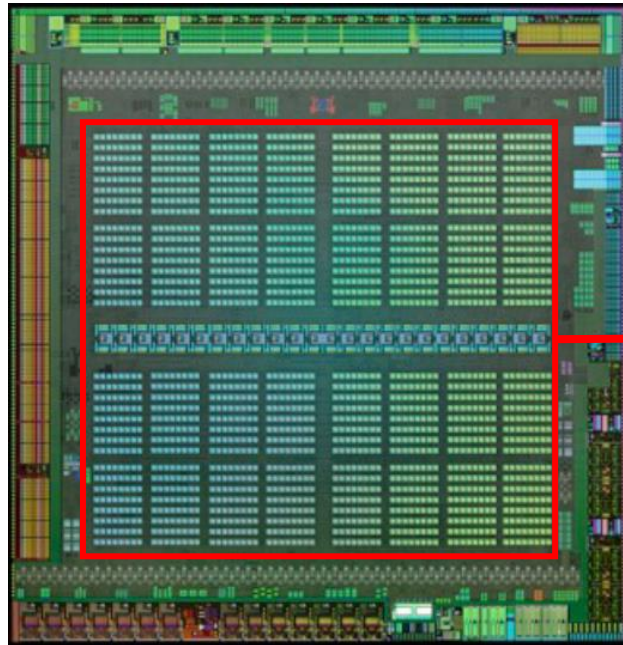


Processor's
Input /outputs

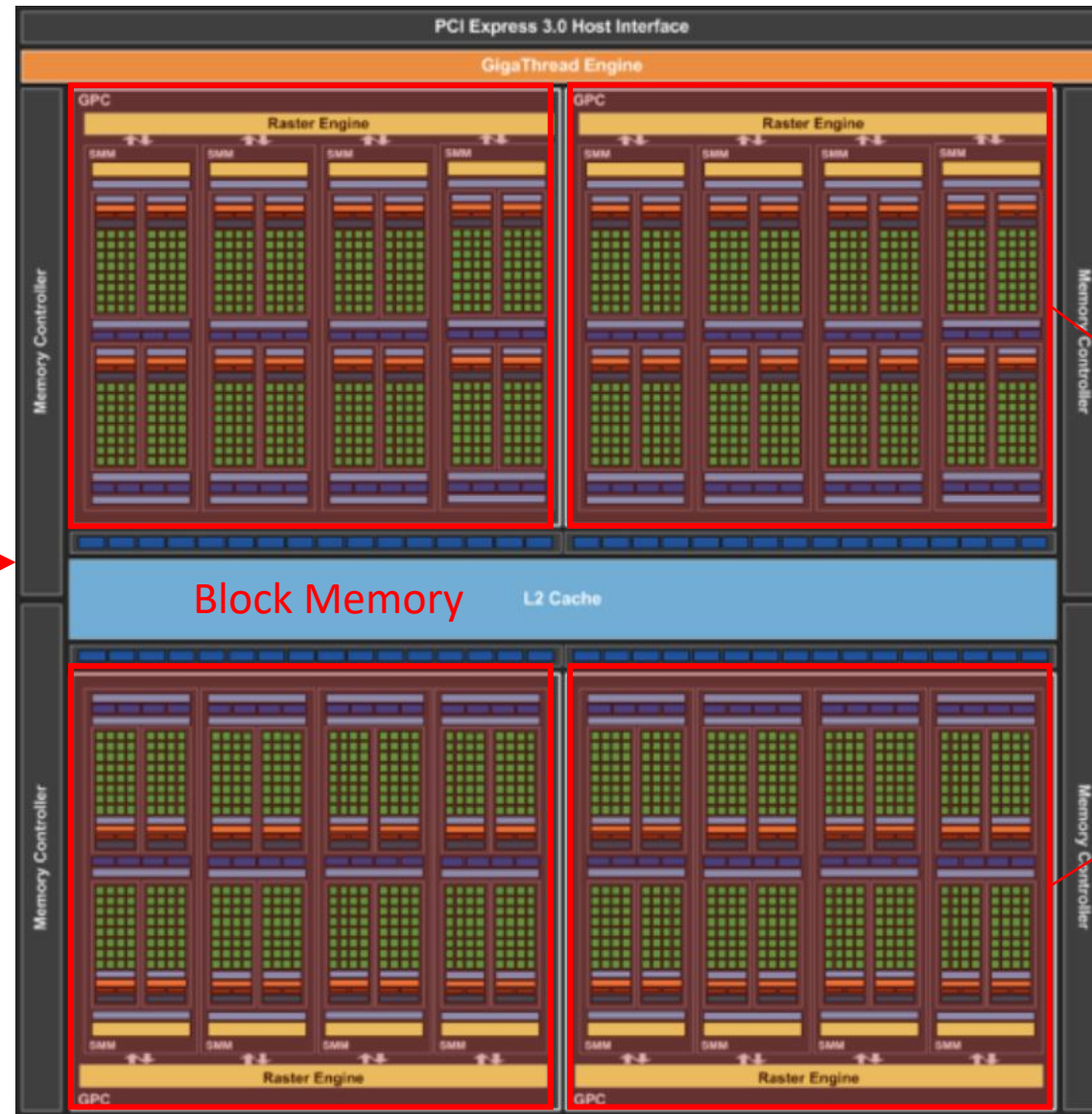


What is a GPU ?

- A processor of processors of threads



Zoom-in

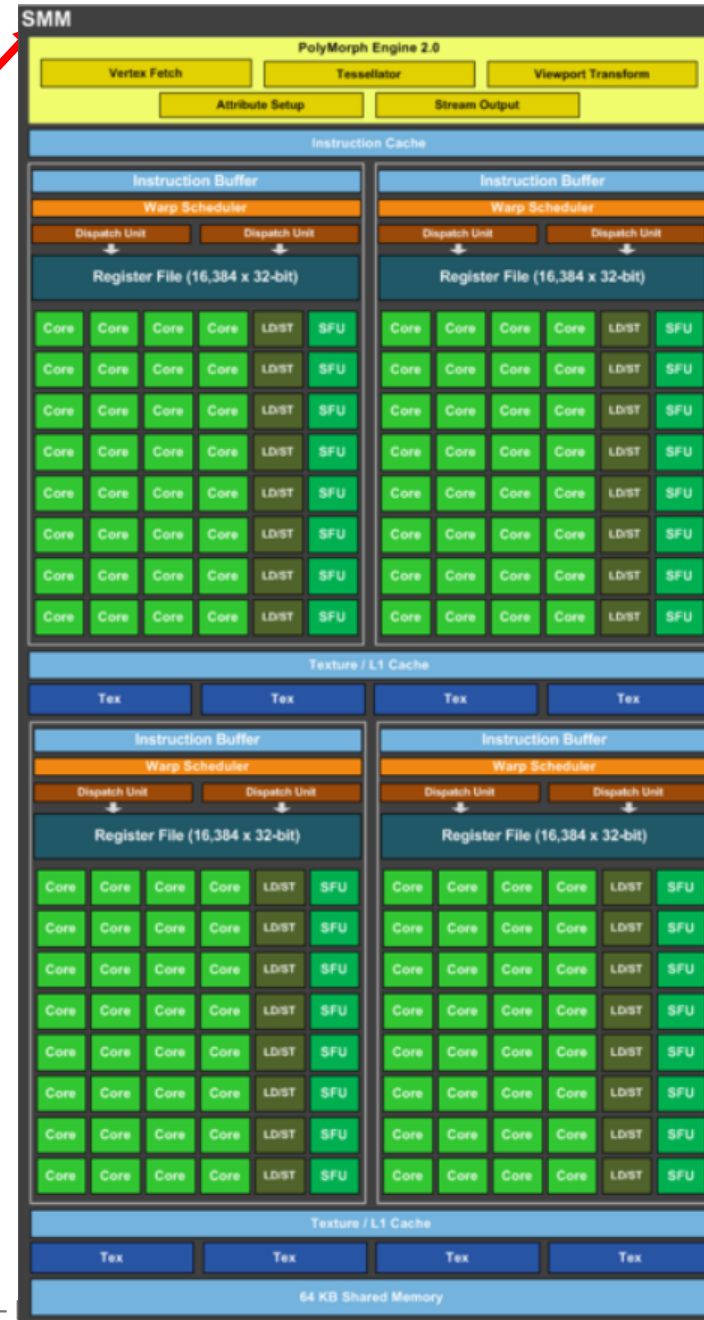
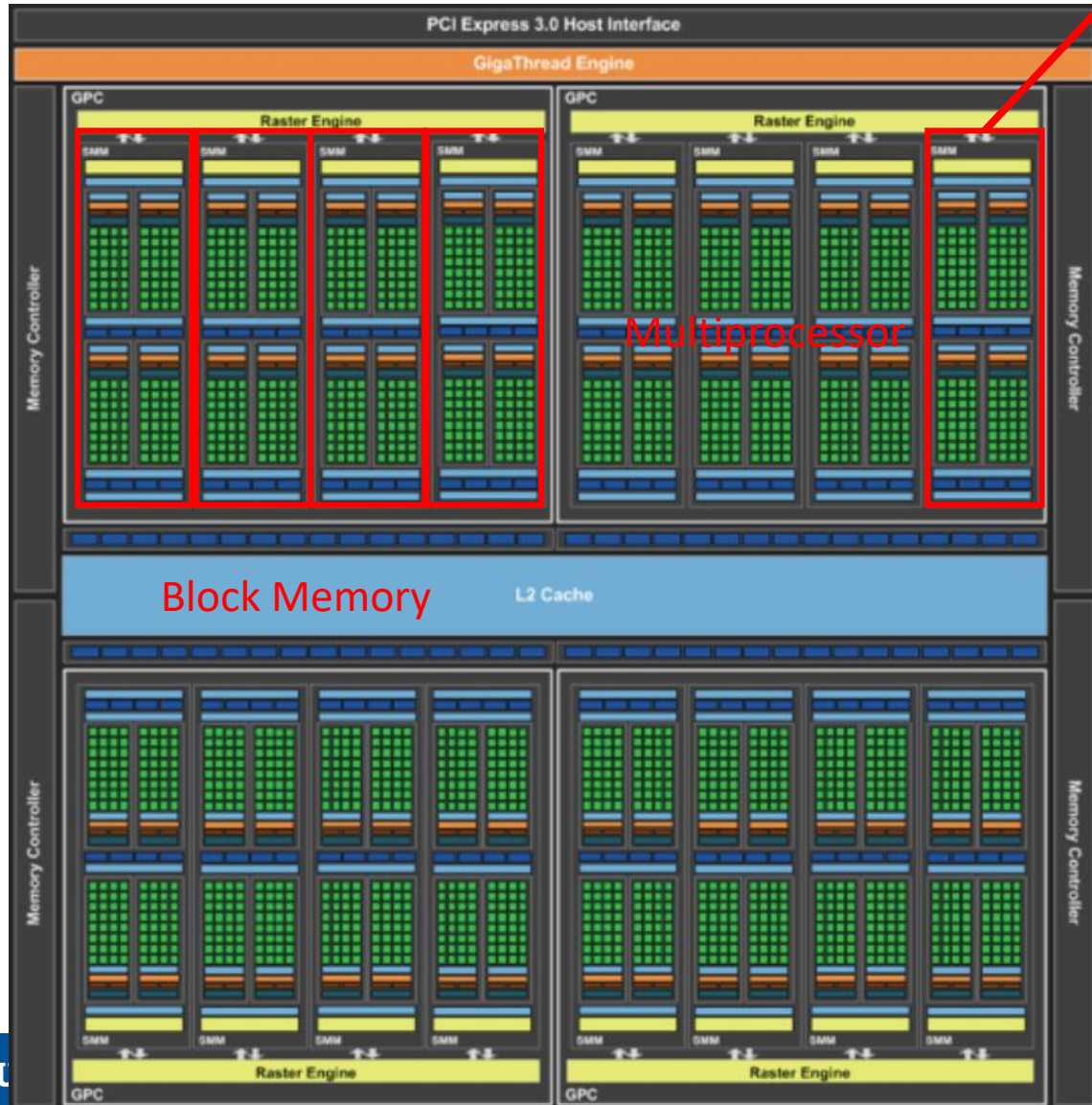


SMMs

=

Multiprocessors
(here: 4x4)

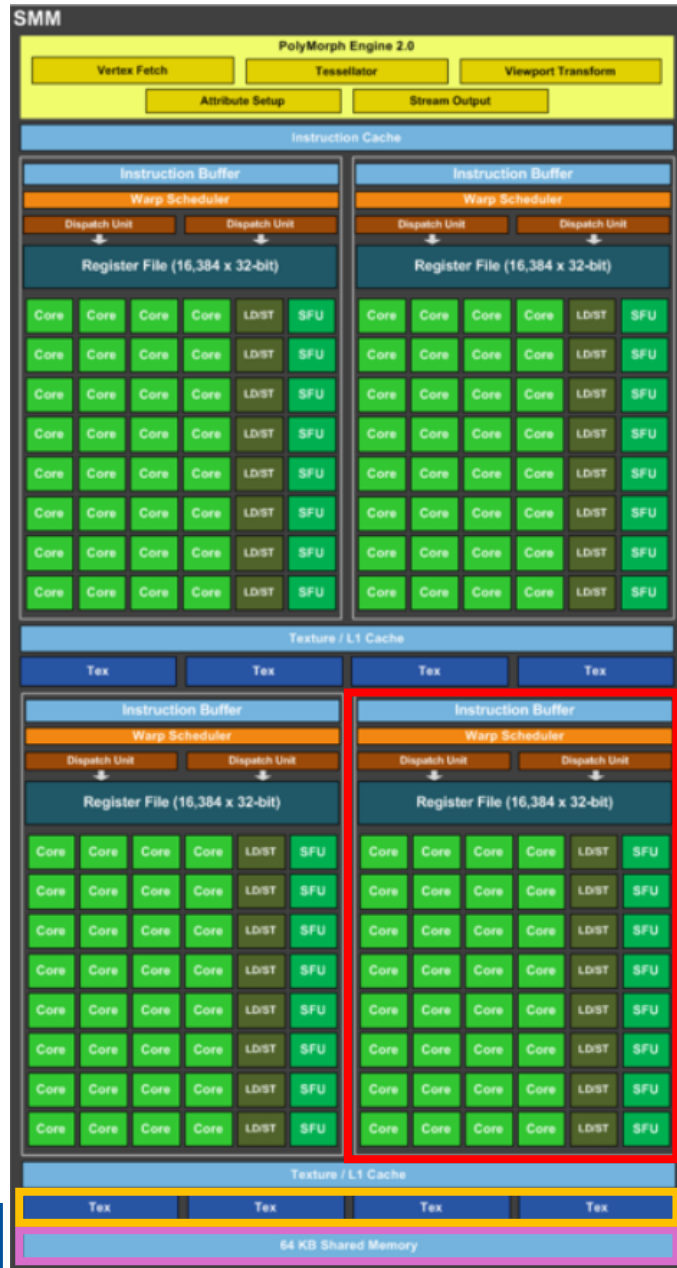
What is a GPU ?



Streaming
Multiprocessor
(SM)

Here: 4 SM
by multiprocessor

What is a GPU ?



Here: 4 WARP
by SM

WARP = 32 threads



Streaming
Multiprocessor
(SM)

Texture memory
Shared memory (SM)

Threads

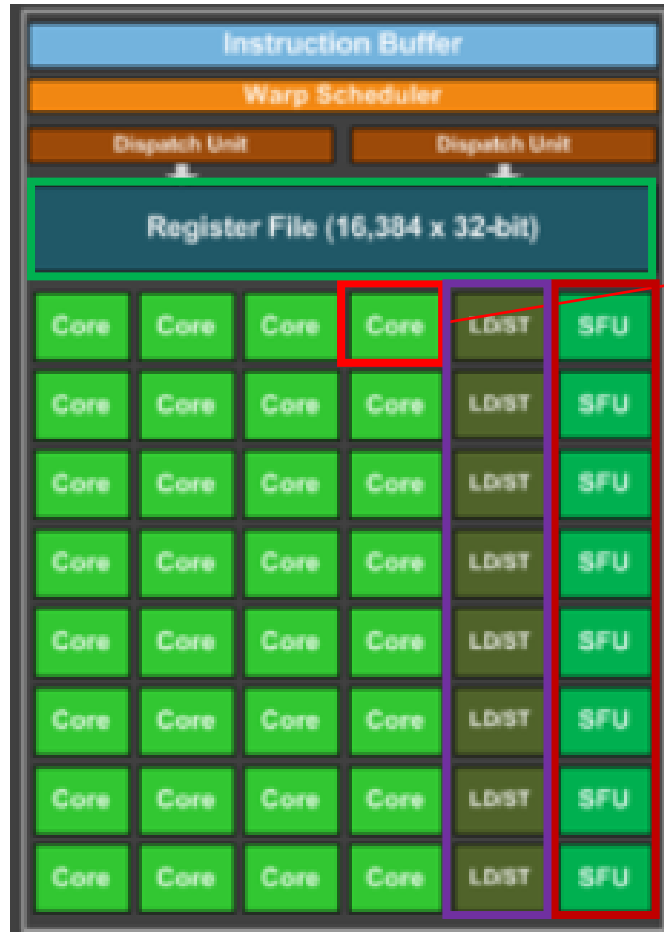
Threads Registers

Load/Store
(core to memory)

Special Function Units
=
fast approximate transcendental
operations
=
sin, exp, ...

What is a GPU ?

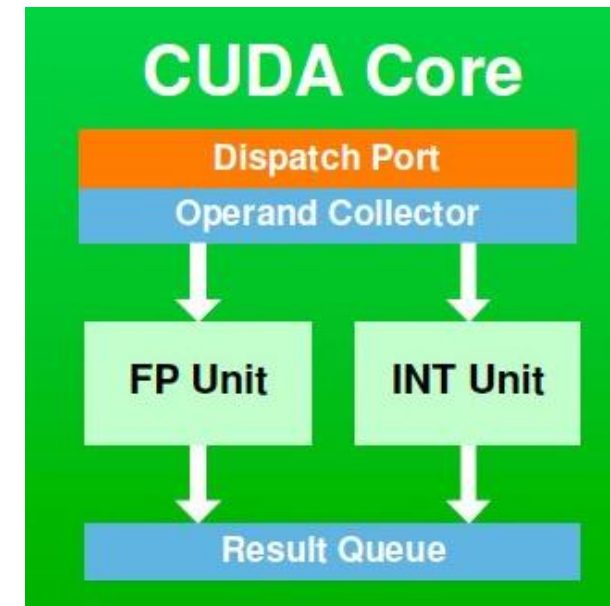
WARP = 32 threads



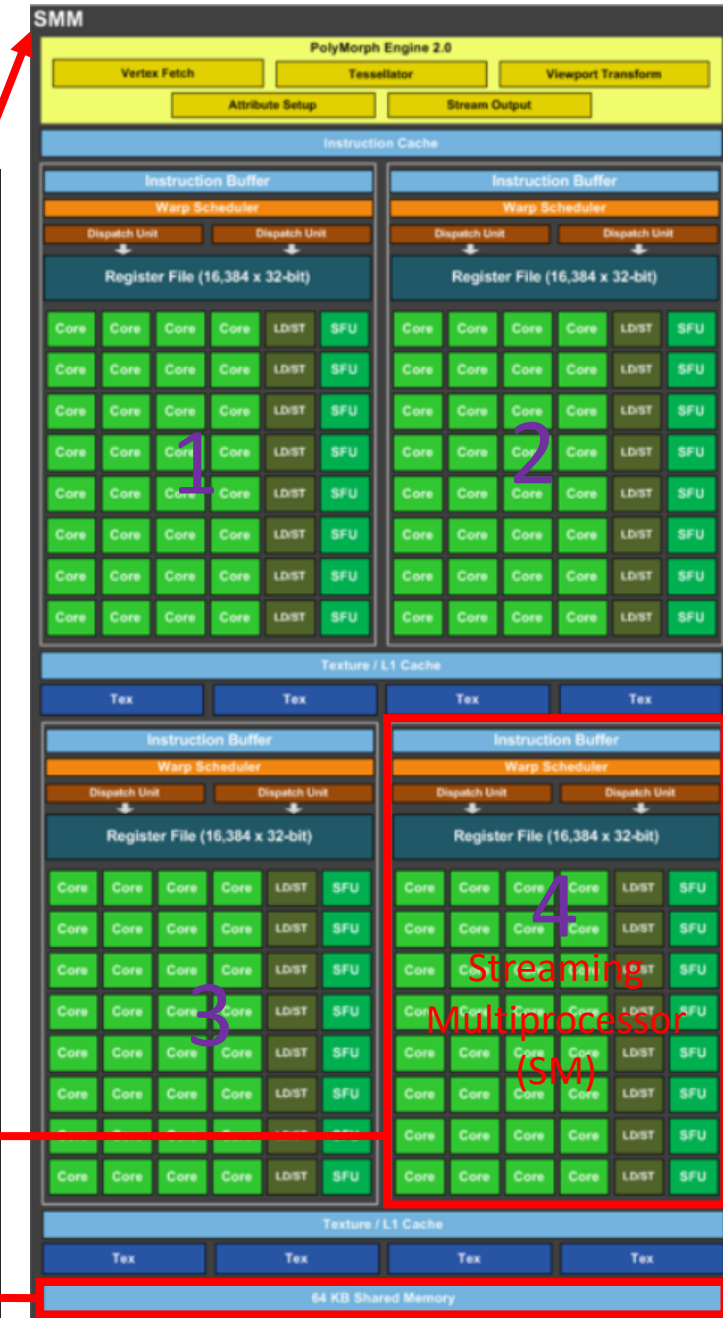
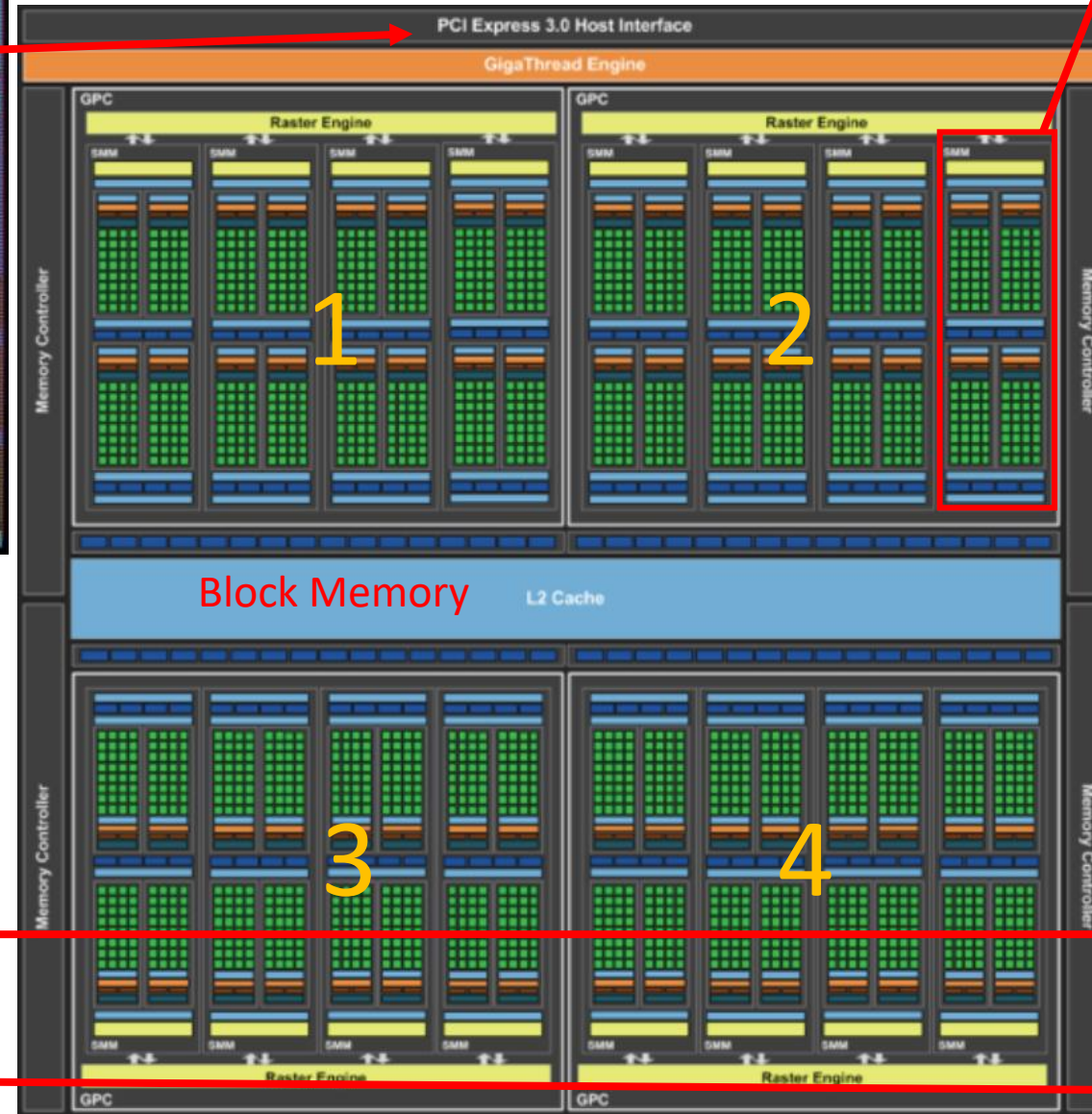
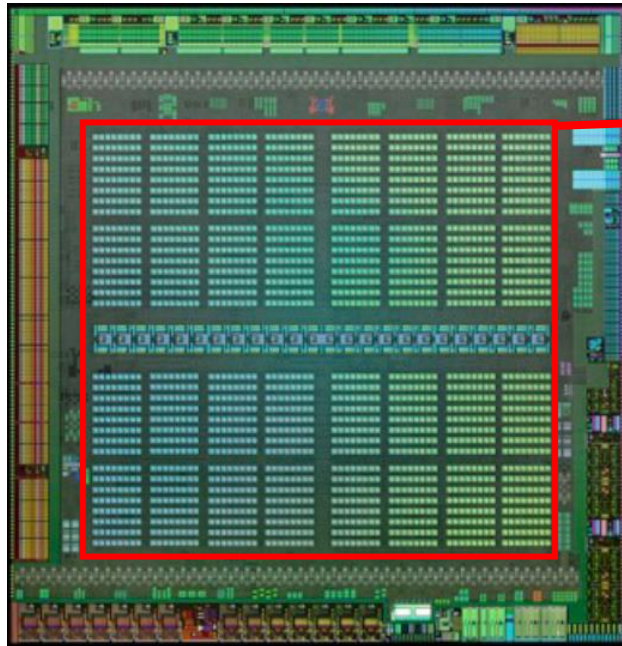
A thread

Floating point unit

Integer Unit



What is a GPU ?



4x4 Multiprocessors
 4 SM (Streaming processor)
 32 Cores per SM
 $(4 \times 4) \times 4 \times 32 = 2048$ cores!

1 thread = 1 core + local memory

Shared memory

Streaming Multiprocessor (SM)

What is a GPU ?

- Characteristics

GeForce	GTX 950	GTX 960	GTX 970	GTX980	GTX 980 Ti	Titan X		GTX 680 (Kepler)	GTX 980 (Maxwell)	GTX 1080 (Pascal)
Release date	Aug'15	Aug'15	Sep'14	Sep'14	Jun'15	Mar'15	Year	2012	2014	2016
GPU (code name)	GM206-250	GM206-300	GM204-200	GM204-400	GM200-310	GM200-400	Transistors	3.54 B @ 28 nm.	5.2 B @ 28 nm.	7.2 B @ 16 nm.
Multiprocessors	6	8	13	16	22	24	Power consumption & die size	195 W & 294 mm ²	165 W & 398 mm ²	180 W & 314 mm ²
Number of cores	768	1024	1664	2048	2816	3072	Multiprocessors	8	16	40
Cores frequency (MHz)	1024-1188	1127-1178	1050-1178	1126-1216	1000-1075	1000-1075	Cores / Multiproc.	192	128	64
DRAM bus width	128 bits	128 bits	256 bits	256 bits	384 bits	384 bits	Cores / GPU	1536	2048	2560
DRAM frequency	2x 3.3 GHz	2x 3.5 GHz	2x 3.5 GHz	2x 3.5 GHz	2x 3.5 GHz	2x 3.5 GHz	Clock (wo. and w. GPU Boost)	1006, 1058 MHz	1126, 1216 MHz	1607, 1733 MHz
DRAM bandwidth	105.6 GB/s	112 GB/s	224 GB/s	224 GB/s	336.5 GB/s	336.5 GB/s	Peak performance	1625 GFLOPS	4980 GFLOPS	8873 GFLOPS
GDDR5 memory size	2 GB	2 GB	4 GB	4 GB	6 GB	12 GB	Shared memory	16, 32, 48 KB	64 KB	
Million of transistors	2940	2940	5200	5200	8000	8000	L1 cache size	48, 32, 16 KB	Integrated with texture cache	
Die size	228 mm ²	228 mm ²	398 mm ²	398 mm ²	601 mm ²	601 mm ²	L2 cache size	512 KB	2048 KB	
Maximum TDP	90 W	120 W	145 W	165 W	250 W	250 W	DRAM memory: Interface	256-bit GDDR5	256-bit GDDR5	256-bit GDDR5X
Power connectors	1 x 6 pines	1 x 6 pines	2 x 6 pines	2 x 6 pines	6 + 8 pines	6 + 8 pines	DRAM memory: Frequency	2x 3000 MHz	2x 3500 MHz	4x 2500 MHz
Price (\$ upon release)	149	199	329	549	649	999	DRAM memory: Bandwidth	192.2 GB/s	224 GB/s	320 GB/s

Often better to have 980 Ti or Titan X than a GTX 1080 in term of computing cores. However, the frequencies/memory not the same!

What is a GPU ?

- Old characteristics (2016-2017)

Lambda users cores evolution:

$$\Delta t = 11 \text{ years}$$

$$\Delta \text{cores} = 2560 - 128 = 2432$$

$$\frac{\Delta \text{cores}}{\Delta t} = 221 \text{ core/year}$$

Professional cards

	Tesla		Fermi		Kepler				Maxwell				Pascal	
Architecture	G80	GT200	GF100	GF104	GK104 (K10)	GK110 (K20X)	GK110 (K40)	GK210 (K80)	GM107 (GTX750)	GM204 (GTX980)	GM200 (Titan X)	GM200 (Tesla M40)	GP104 (GeForce GTX 1080)	GP100 (Tesla P100)
Time frame	2006 /07	2008 /09	2010	2011	2012	2013	2013 /14	2014	2014 /15	2014 /15	2016	2016	2016	2017
CUDA Compute Capability	1.0	1.3	2.0	2.1	3.0	3.5	3.5	3.7	5.0	5.2	5.3	5.3	6.0	6.0
N (multiprocs.)	16	30	16	7	8	14	15	30	5	16	24	24	40	56
M (cores/multip.)	8	8	32	48	192	192	192	192	128	128	128	128	64	64
Number of cores	128	240	512	336	1536	2688	2880	5760	640	2048	3072	3072	2560	3584

The number of cores have increased in a few years!

What is a GPU ?

- 2016 vs 2020
- Advent of:
 - Deep Learning
 - Cryptocurrencies mining

GeForce	GTX 950	GTX 960	GTX 970	GTX980	GTX 980 Ti	Titan X
Release date	Aug'15	Aug'15	Sep'14	Sep'14	Jun'15	Mar'15
GPU (code name)	GM206-250	GM206-300	GM204-200	GM204-400	GM200-310	GM200-400
Multiprocessors	6	8	13	16	22	24
Number of cores	768	1024	1664	2048	2816	3072
Cores frequency (MHz)	1024-1188	1127-1178	1050-1178	1126-1216	1000-1075	1000-1075
DRAM bus width	128 bits	128 bits	256 bits	256 bits	384 bits	384 bits
DRAM frequency	2x 3.3 GHz	2x 3.5 GHz	2x 3.5 GHz	2x 3.5 GHz	2x 3.5 GHz	2x 3.5 GHz
DRAM bandwidth	105.6 GB/s	112 GB/s	224 GB/s	224 GB/s	336.5 GB/s	336.5 GB/s
GDDR5 memory size	2 GB	2 GB	4 GB	4 GB	6 GB	12 GB
Million of transistors	2940	2940	5200	5200	8000	8000
Die size	228 mm ²	228 mm ²	398 mm ²	398 mm ²	601 mm ²	601 mm ²
Maximum TDP	90 W	120 W	145 W	165 W	250 W	250 W
Power connectors	1 x 6 pines	1 x 6 pines	2 x 6 pines	2 x 6 pines	6 + 8 pines	6 + 8 pines
Price (\$ upon release)	149	199	329	549	649	999

$$\Delta t = 5 \text{ years}$$

$$\Delta \text{cores} = 10496 - 3072 = 7424$$

$$\frac{\Delta \text{cores}}{\Delta t} = 1485 \text{ core/year}$$

Lambda users cores evolution:

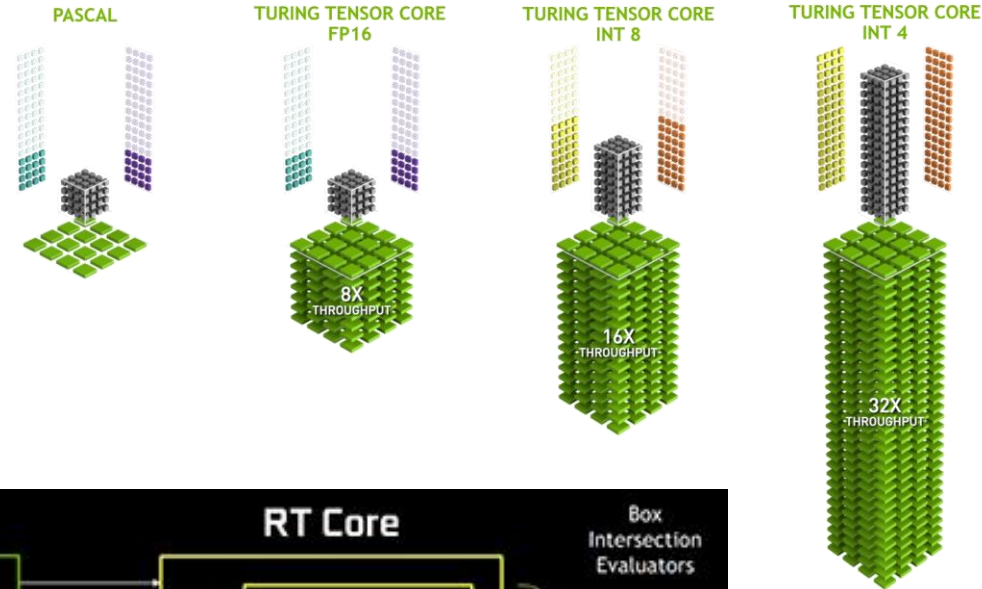
NEW KIND OF CORES:

- Tensor cores for Deep Learning
- RT Cores for Ray Tracing

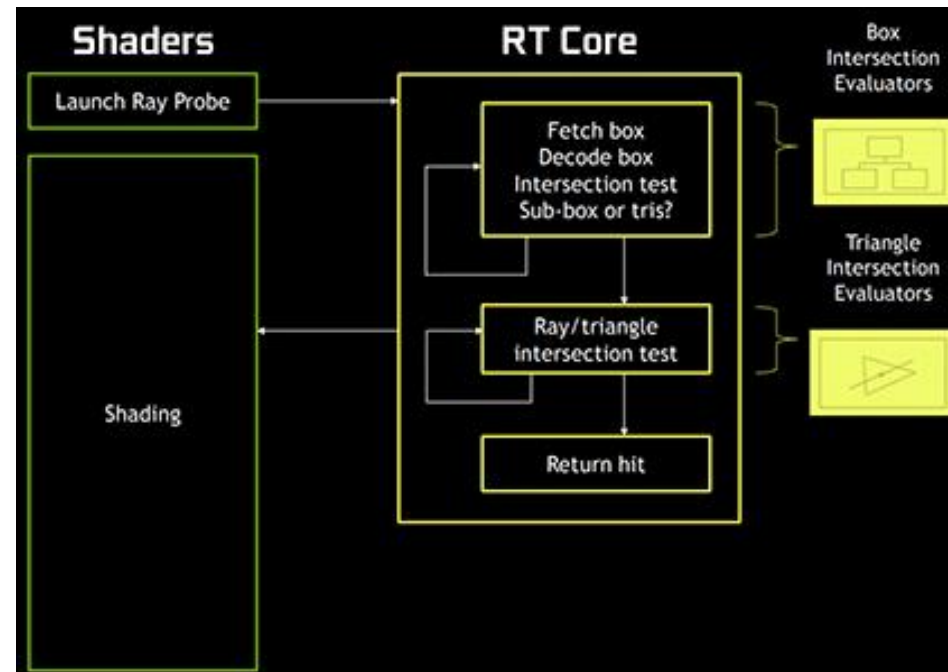
NVIDIA GeForce RTX 30 Series RUMORED Specifications					
VideoCardz.com	RTX 3090	RTX 3080	TBC	RTX 3070	RTX 3060 Ti
Picture			[To be leaked]		[To be leaked]
Reference Board	PG132 SKU 30	PG132 SKU 10	TBC	PG142 SKU 10	PG190 SKU 10
GPU	8nm GA102-300	8nm GA102-200	8nm GA102-150	8nm GA104-300	8nm GA104-200
Die Size	628 mm ²	628 mm ²	628 mm ²	392 mm ²	392 mm ²
Transistors	28.3 B	28.3 B	28.3 B	17.4 B	17.4 B
CUDA Cores	10496	8704	7424	5888	4864
GPU Utilization	98%	81%	70%	96%	79%
Tensor Cores	328	272	232	184	152
RT Cores	82	68	58	46	38
Base Clock	1395 MHz	1440 MHz	TBC	1500 MHz	TBC
Boost Clock	1695 MHz	1710 MHz	TBC	1725 MHz	TBC
Shader Perf.	35.6 TFLOPS	29.8 TFLOPS	TBC	20.3 TFLOPS	TBC
Memory	24 GB G6X	10 GB G6X	10 GB G6X	8 GB G6	8 GB G6
Memory Clock	19.5 Gbps	19 Gbps	TBC	14 Gbps	14 Gbps
Memory Bus	384-bit	320-bit	320-bit	256-bit	256-bit
Max Bandwidth	936 GB/s	760 GB/s	TBC	448 GB/s	448 GB/s
Thermal Design	350W	320W	TBC	220W	~180W
MSRP	\$1,499	\$699	TBC	\$499	TBC
Release Date	September 24th	September 17th	TBC	October 29th	Mid November

What is a GPU ?

- **Ray tracing** (INFO-H502) and Tensor cores (Deep Learning)



Tensor Core:
- Fast at **convolutions**



RT Core:
- **Ray primitives**
- **Fast intersections**

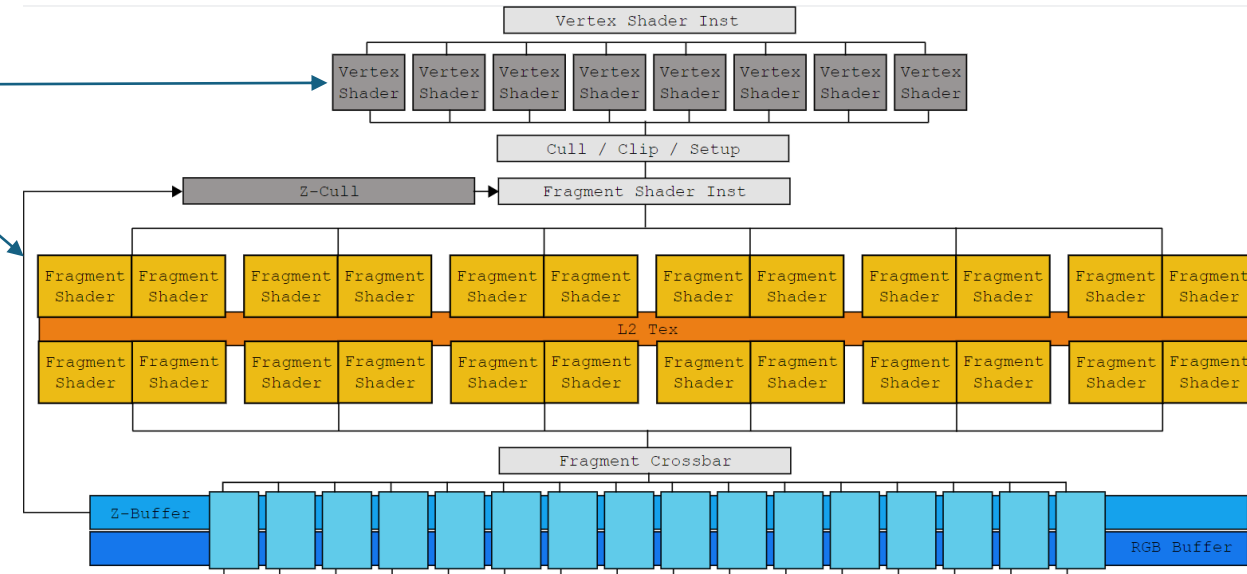
- Bounding box
- Triangle

What is a GPU ?

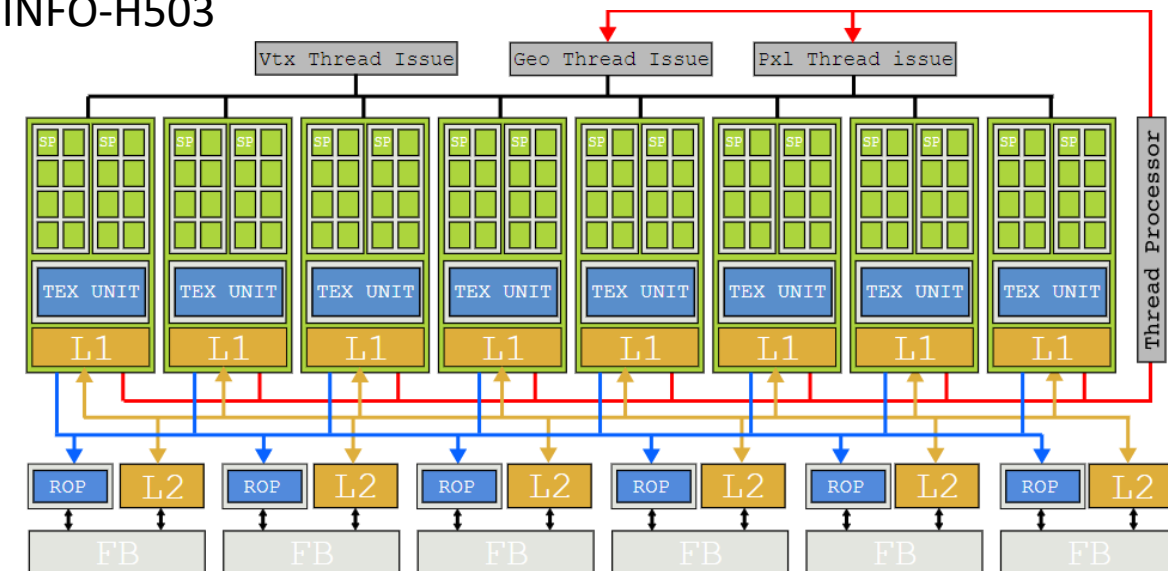
- Global evolution:

- GPUs changed in 2006 from **Graphic Processing Units** to **General Processing Units**
- They still perform both: Graphic and Computing operations
- “CUDA Interop” is when you make CUDA and OpenGL work together
 - CUDA write in texture space (a buffer)
 - OpenGL display this texture when it is ready
- If you want to dig deeper in this evolution:
 - <https://fabiansanglard.net/cuda/index.html>
 - Is a perfect start (also in the PDFs)
 - Sanglard - 2020 - A history of NVidia Stream Multiprocessor

INFO-H502

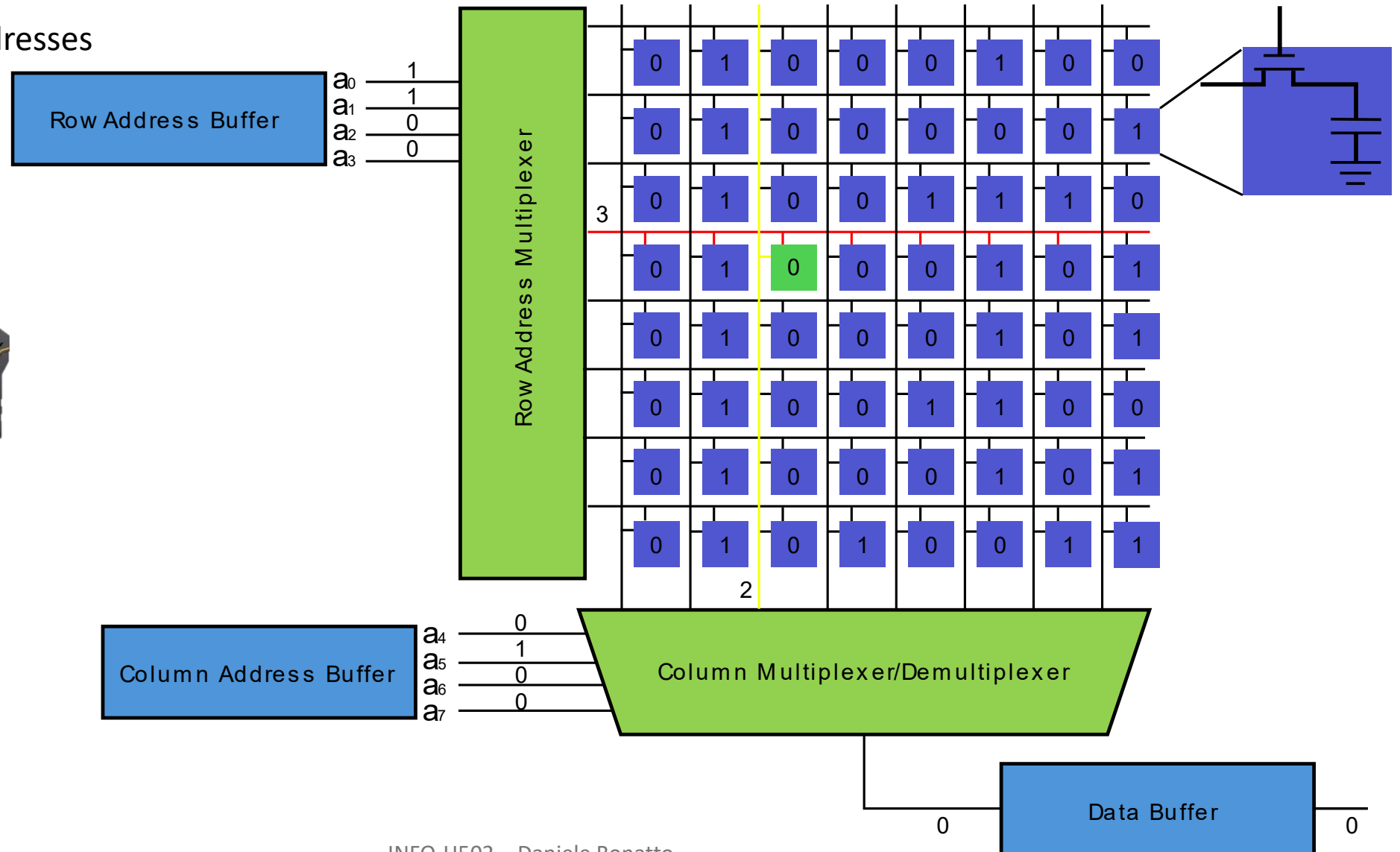


INFO-H503



Random Access Memory (RAM) – CPU side

$2^8 = 256$ possible addresses

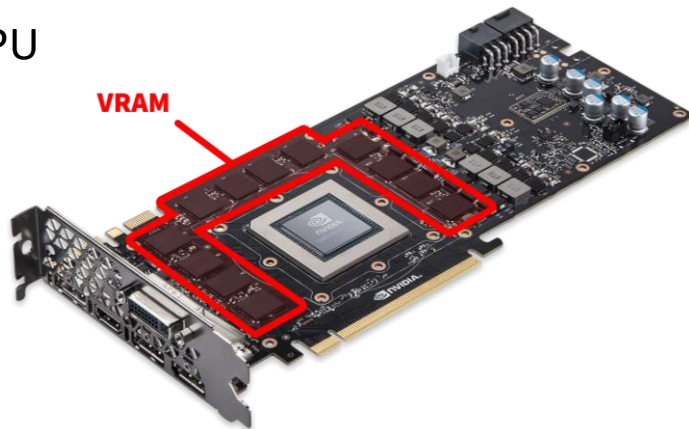


Random Access Memory (RAM) – Video RAM

Programmer perspective



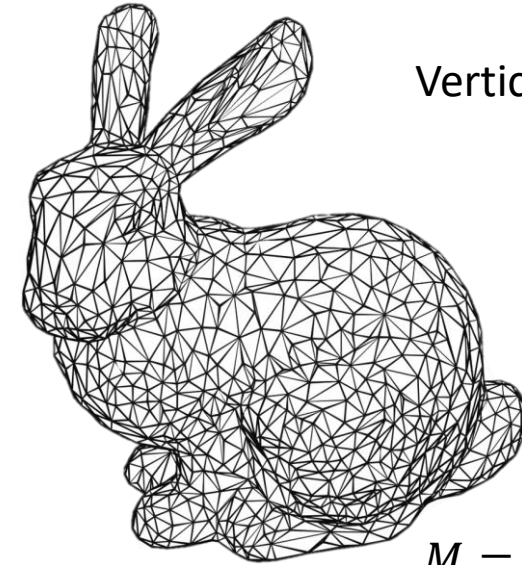
GPU



What can we store ?

Address	Value
0x00	01000100
0x01	01000010
0x02	01001110
0x03	01001001
0x04	01000101
0x05	01001100
0x06	01000101
0x07	01010011

Vertices



Values

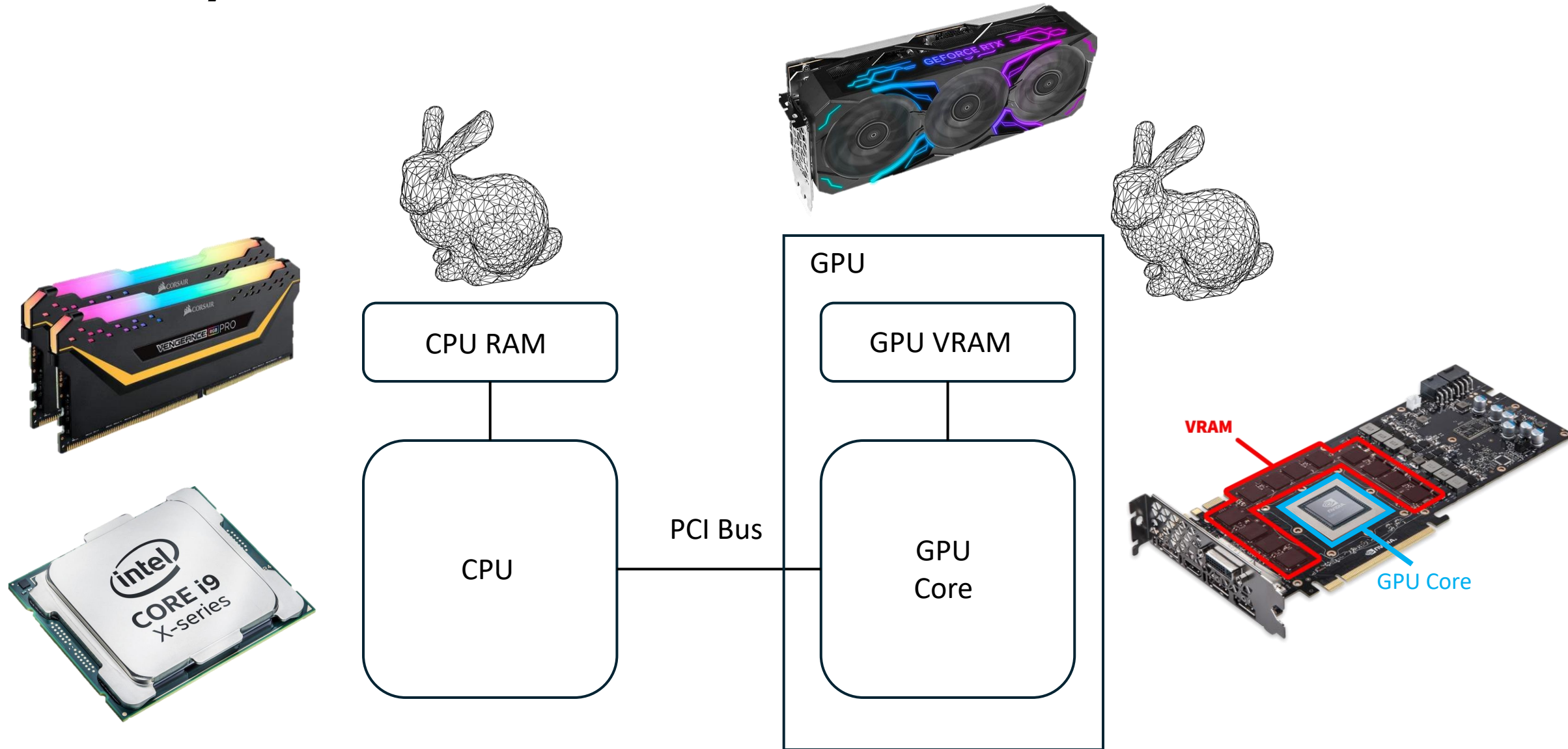
$$M = \begin{pmatrix} 1 & 20 & 5 \\ 24 & 1 & 65 \\ 45 & 11 & 1 \end{pmatrix}$$

Images



Etc.

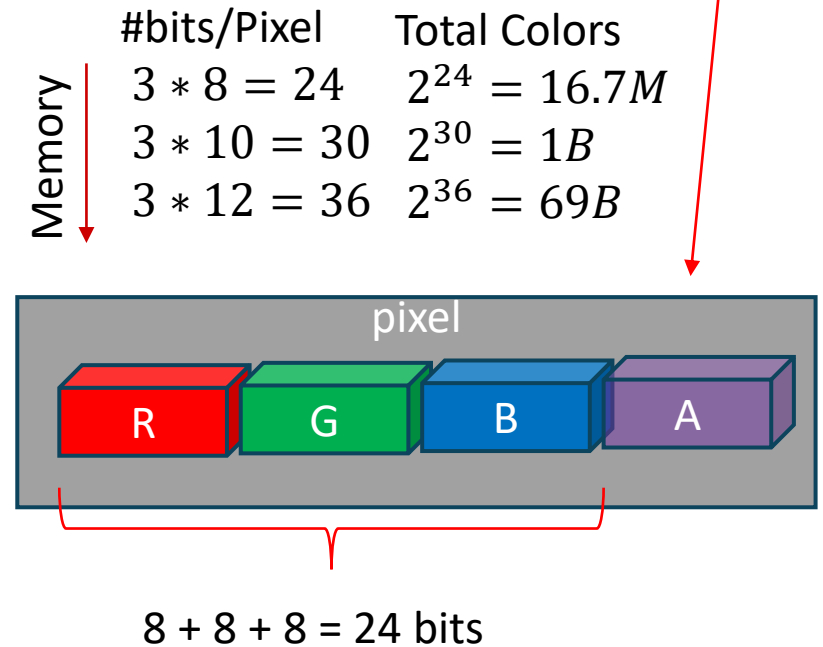
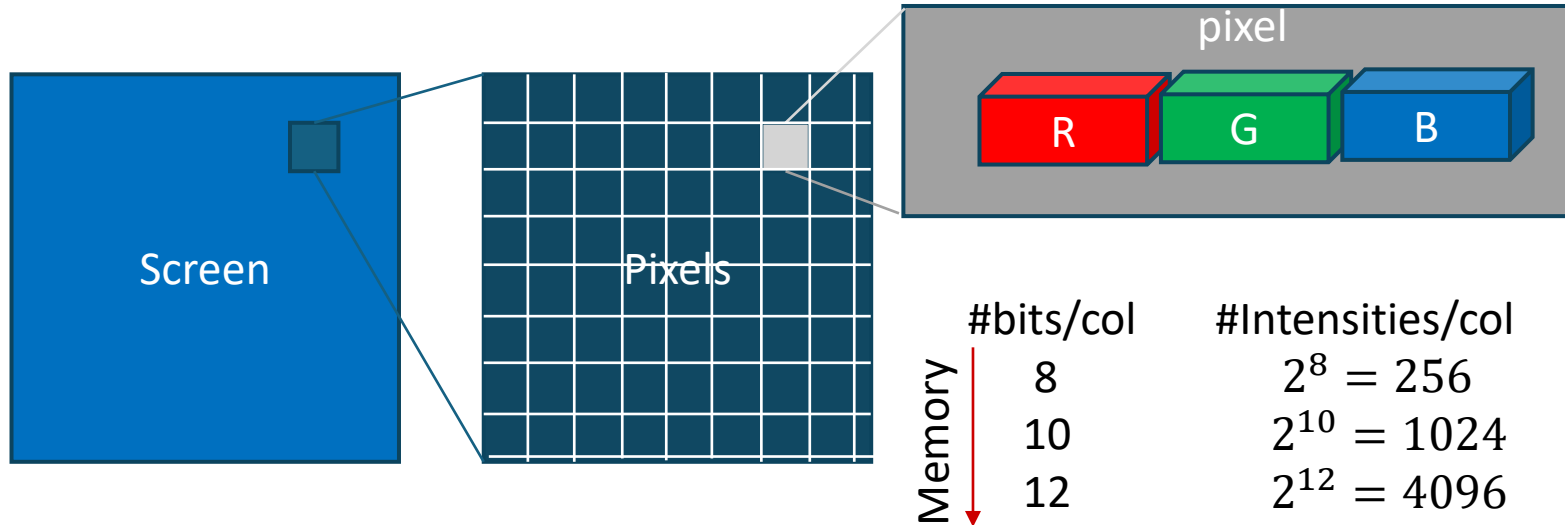
Memory Transfers – RAM – Video RAM



VRAM – Buffers

- What is a buffer ? What do they contain ?

Adding alpha
makes a vector
more memory
friendly!

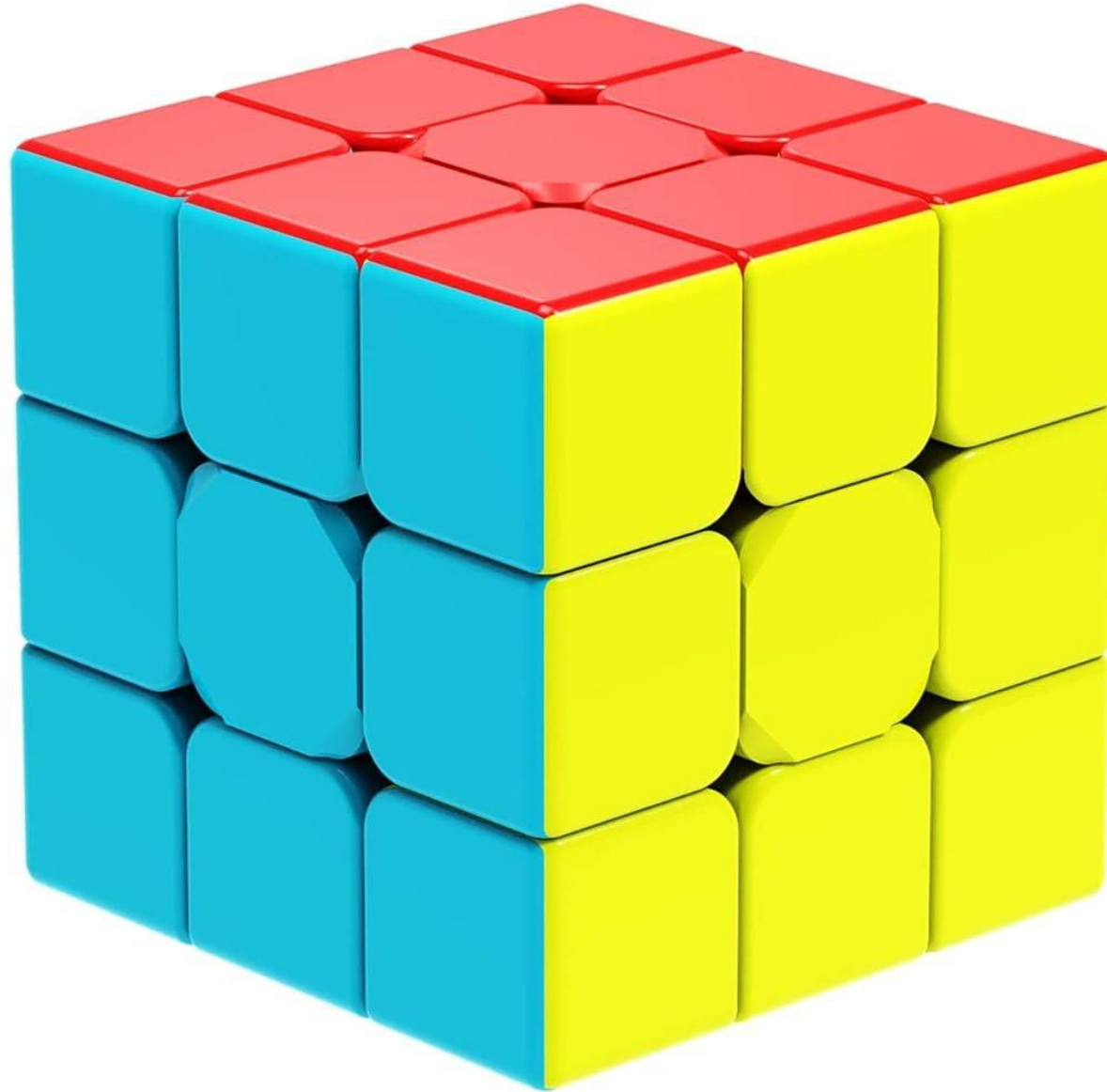


Slides on buffers, colors, alpha component, double-buffers, lights and Rasterisation inspired by:
SIGGRAPH University 2014 – Fundamentals Seminar by Mike Bailey (Oregon State University)

- In INFO-H502 we use GLSL



Drawing a cube – In class exercise



OpenGL Pipeline

- Pipeline transforms **input objects** into final **rendering**

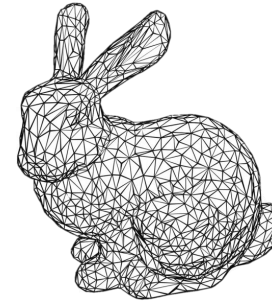
3D objects (meshes), Textures, Matrices, ...

Colors on a buffer/screen

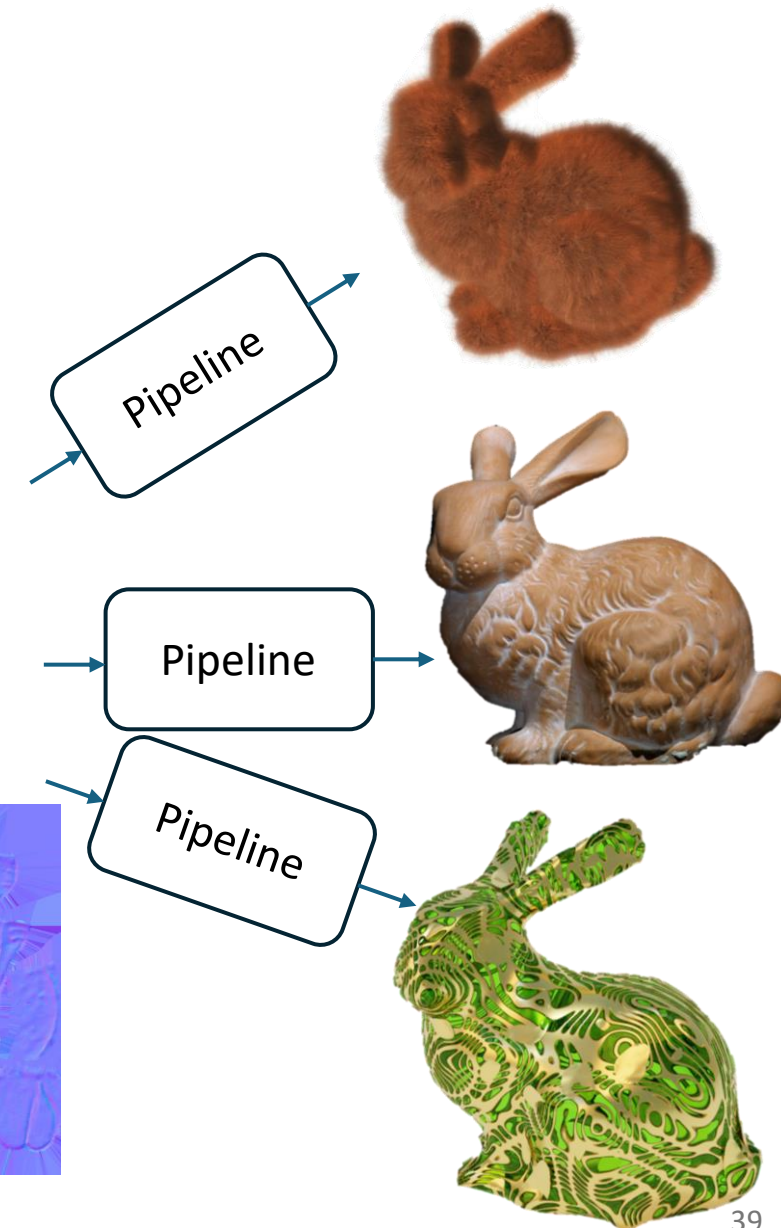
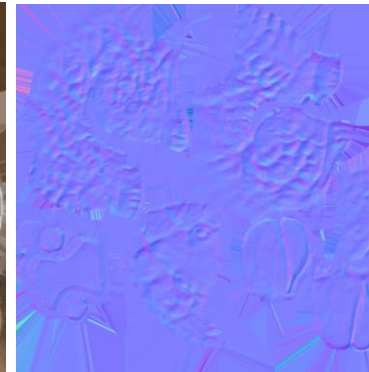
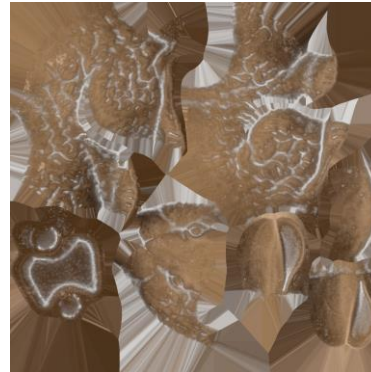
- One pipeline = one rendering effect

- To do that, we:

- Prepare the data on the CPU
- Write shaders for a pipeline
- Apply the pipeline
- Show the result on screen



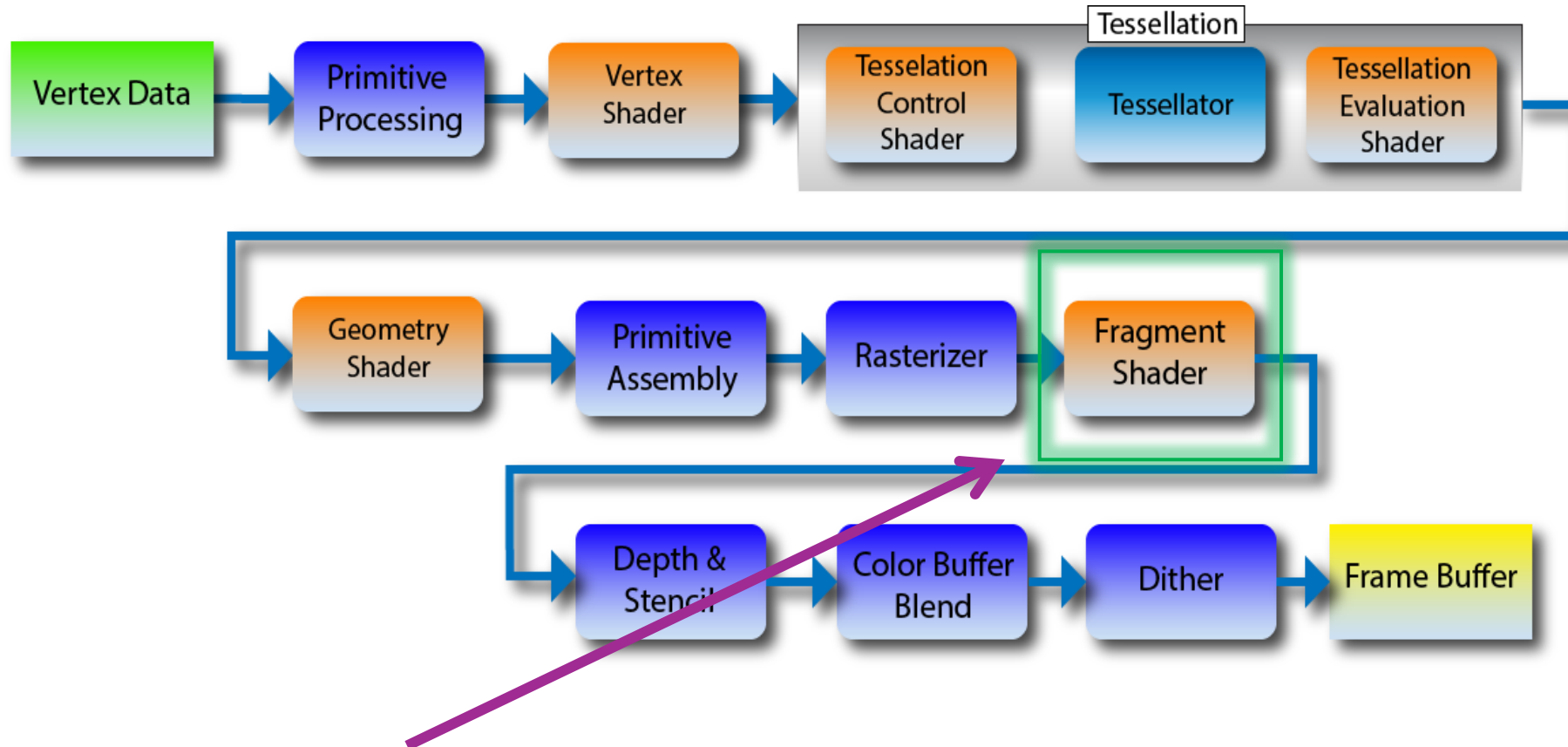
$$M = \begin{pmatrix} 1 & 20 & 5 \\ 24 & 1 & 65 \\ 45 & 11 & 1 \end{pmatrix}$$



OpenGL Pipeline

How to learn this?

Main difficulty: You need to understand the whole to make an application



First TP session, we focus on Fragment Shader

OpenGL Pipeline

Input data: Vertices + Edges

Vertex Normal Attributes (eg: vertex color)

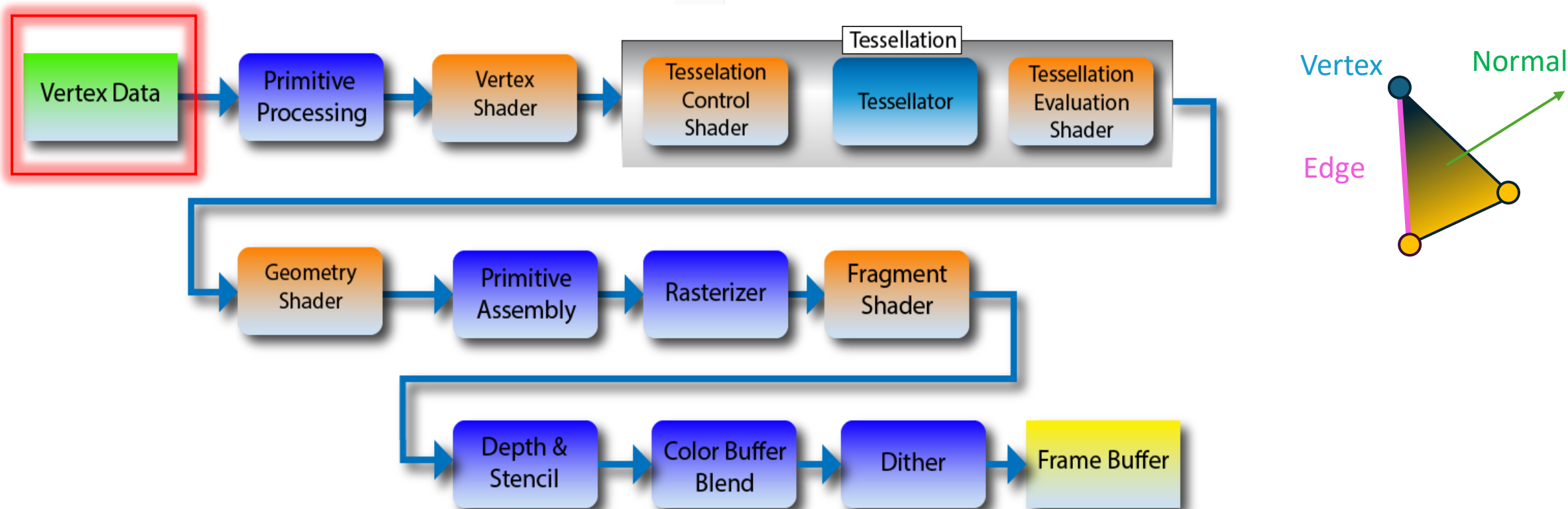
bunny.mesh 447 KB

```

1 4876
2 0.00273012 -0.75114 0.445136 -0.0934739 -0.751342 0.42608 -0.0161479 -0.704623 0.359556 1 1 1
3 0.19653 -0.748571 0.439299 0.127738 -0.739661 0.449994 0.190269 -0.690756 0.389317 1 1 1
4 0.275476 -0.730451 0.400454 0.269782 -0.69383 0.331669 0.370418 -0.689443 0.341495 1 1 1
5 0.370418 -0.689443 0.341495 0.453109 -0.690704 0.301089 0.466236 -0.742544 0.349141 1 1 1
6 0.370418 -0.689443 0.341495 0.381449 -0.749945 0.385336 0.336162 -0.750407 0.40263 1 1 1
7 0.00273012 -0.75114 0.445136 -0.0161479 -0.704623 0.359556 0.0478671 -0.694524 0.383326 1 1 1
8 0.336162 -0.750407 0.40263 0.275476 -0.730451 0.400454 0.370418 -0.689443 0.341495 1 1 1
9 0.370418 -0.689443 0.341495 0.466236 -0.742544 0.349141 0.381449 -0.749945 0.385336 1 1 1
10 0.127738 -0.739661 0.449994 0.00273012 -0.75114 0.445136 0.0478671 -0.694524 0.383326 1 1 1

```

3 vertices = one triangle



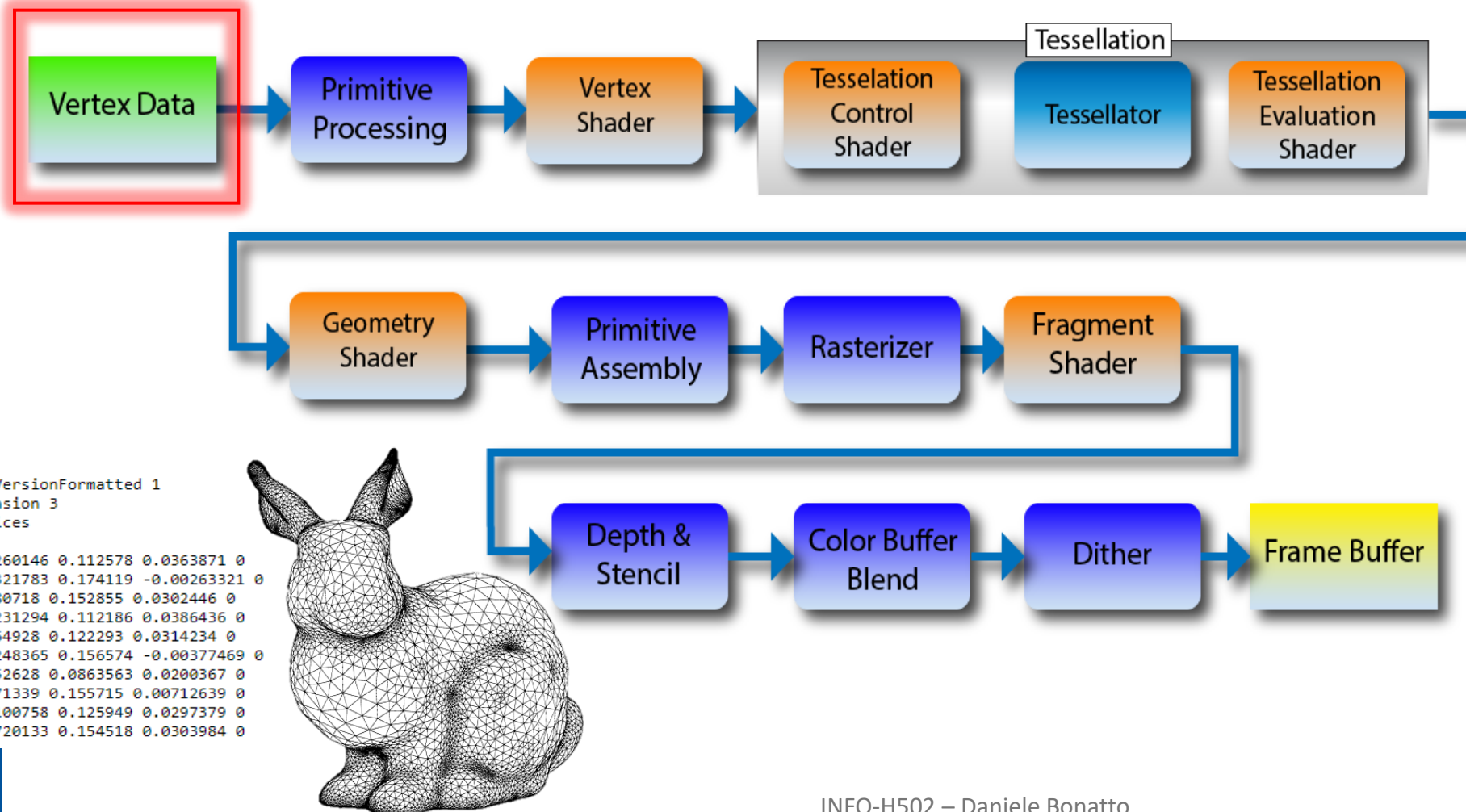
How to generate this data?

OpenGL Pipeline

Vertices usually comes from 3D softwares
Graphic modeling Triangles with attributes



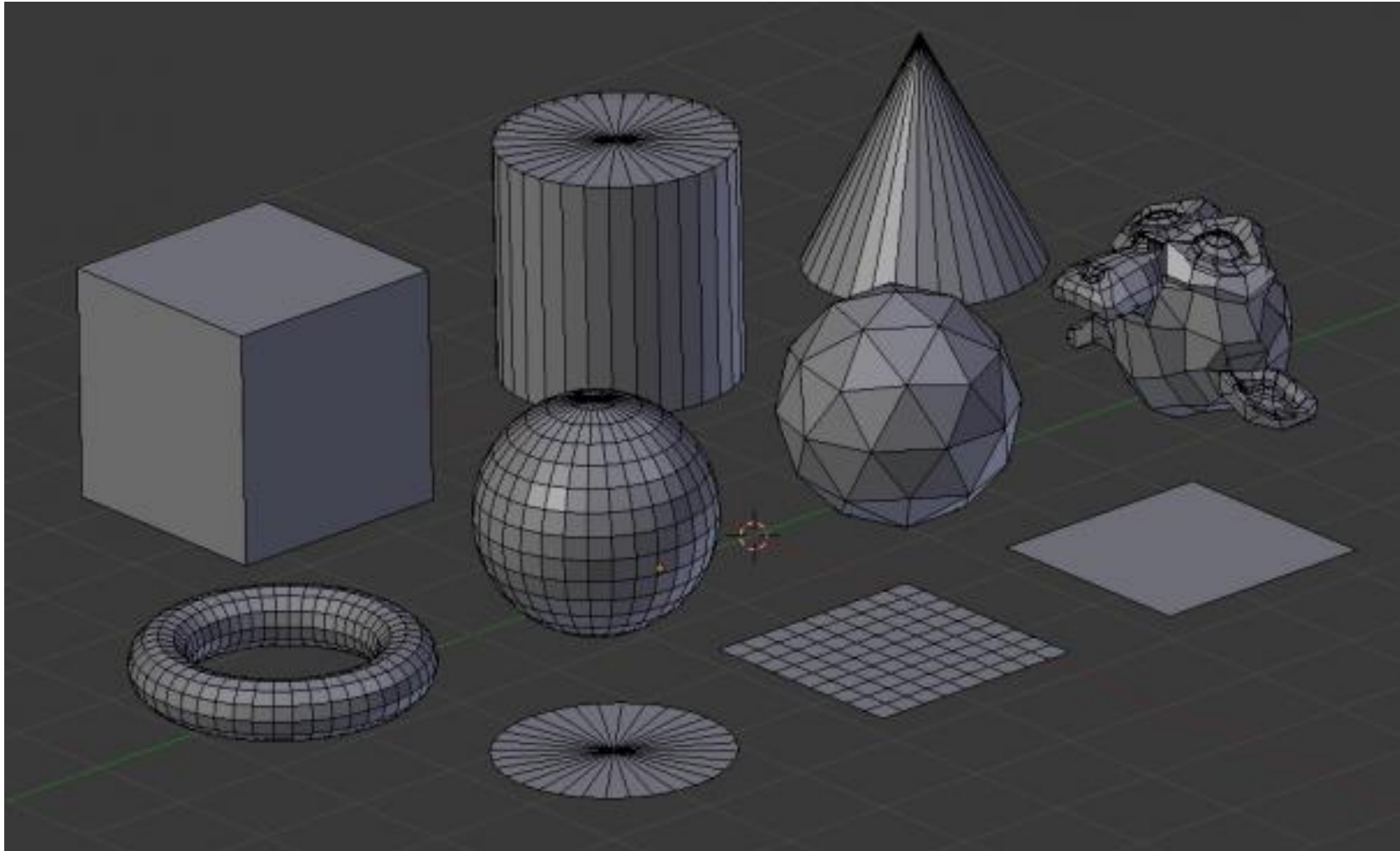
- Free
- Open-source
- Easy to use
- Tutorials!



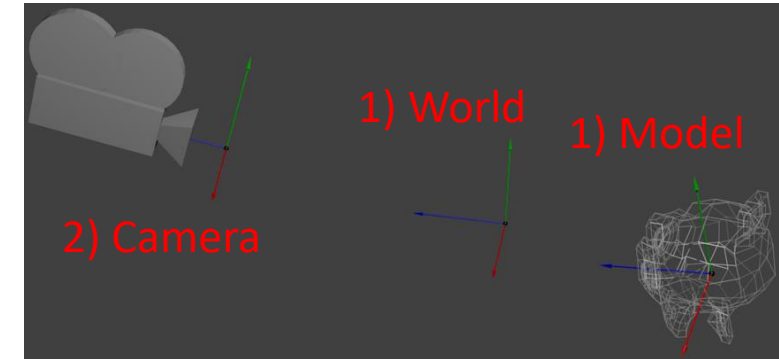
```
MeshVersionFormatted 1
Dimension 3
Vertices
5433
-0.0260146 0.112578 0.0363871 0
-0.0321783 0.174119 -0.00263321 0
-0.080718 0.152855 0.0302446 0
-0.0231294 0.112186 0.0386436 0
0.0164928 0.122293 0.0314234 0
-0.0248365 0.156574 -0.00377469 0
0.0452628 0.0863563 0.0200367 0
-0.071339 0.155715 0.00712639 0
-0.0100758 0.125949 0.0297379 0
-0.0720133 0.154518 0.0303984 0
```

OpenGL Pipeline

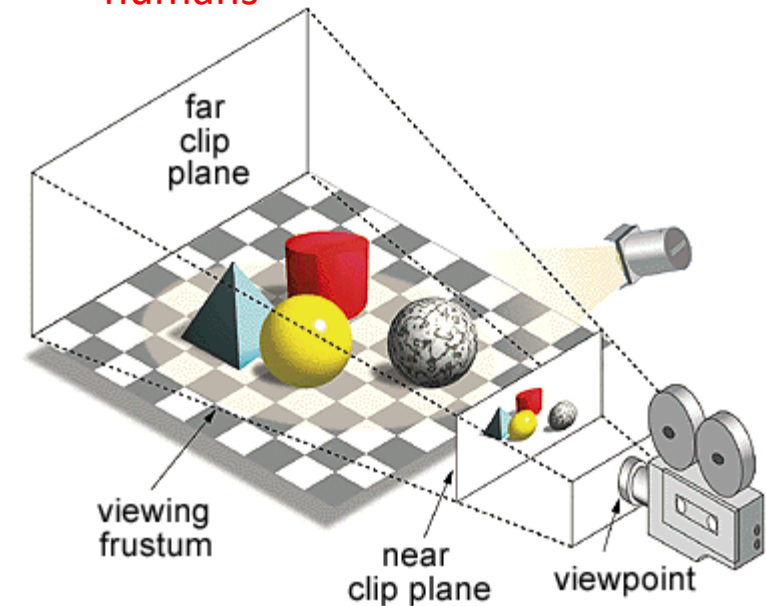
1) We need to move the objects to make a scene



2) We want to look with the camera

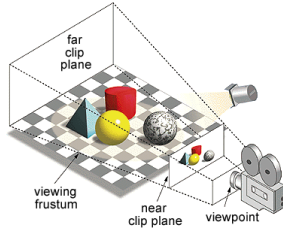


3) We need to add a projection to make them realistic for humans

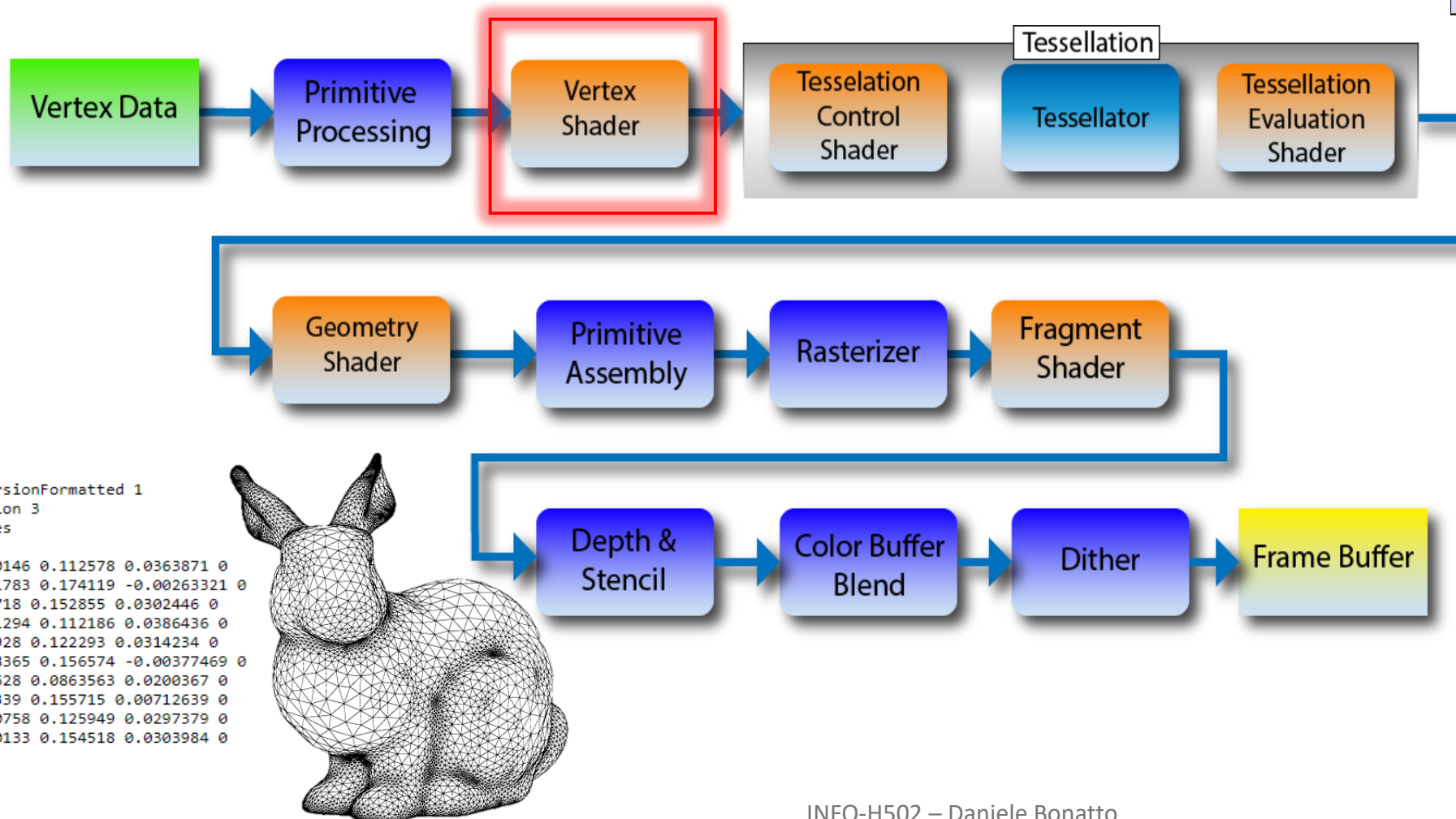
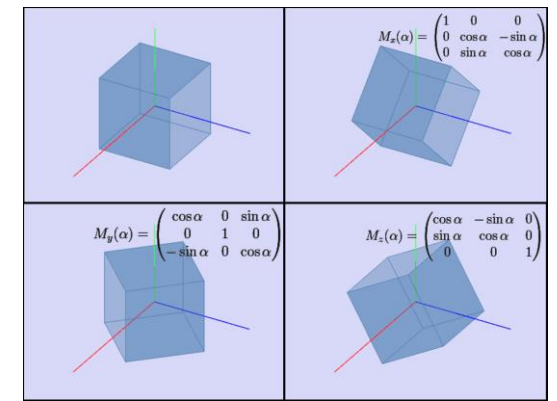


How ?

OpenGL Pipeline



Linear Algebra: Matrix transformations – Lecture 2, 3
Model, Camera, Projection spaces – Lecture 4

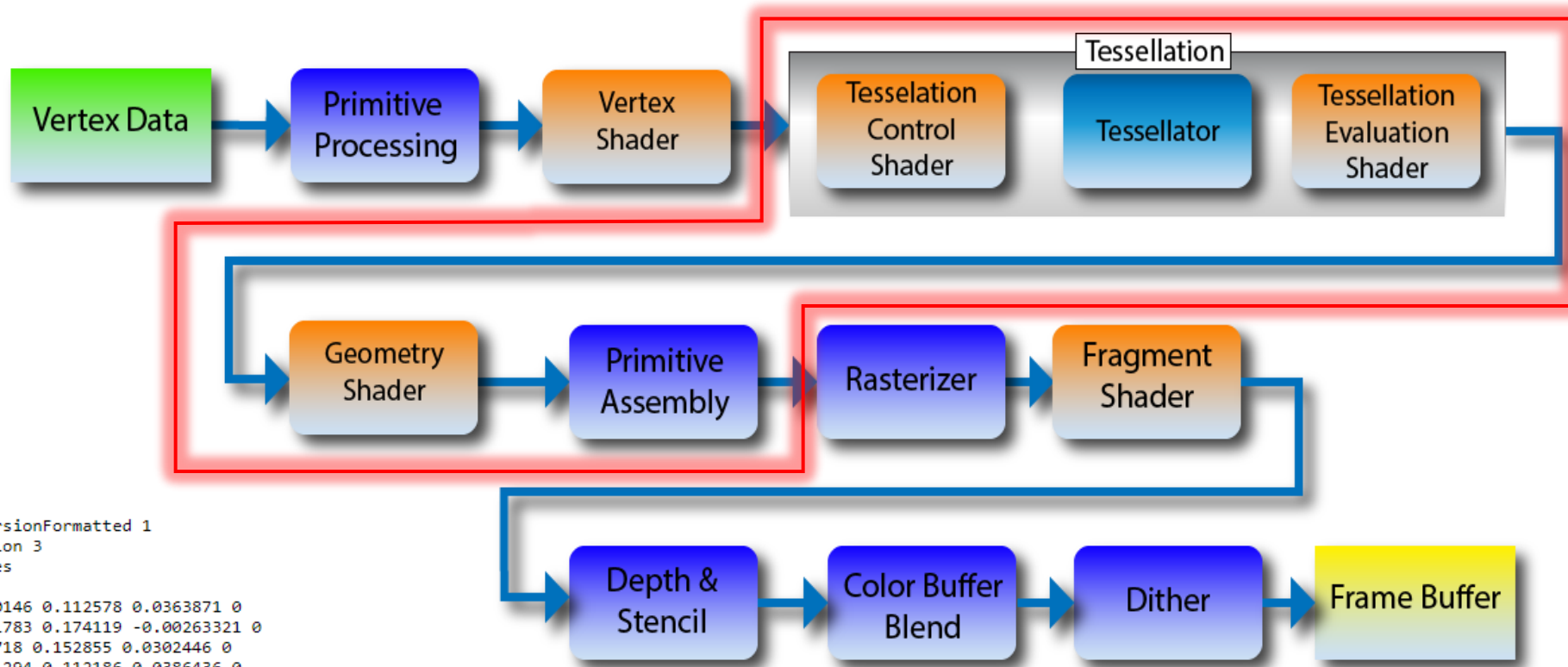


```

MeshVersionFormatted 1
Dimension 3
Vertices
5433
-0.0260146 0.112578 0.0363871 0
-0.0321783 0.174119 -0.00263321 0
-0.080718 0.152855 0.0302446 0
-0.0231294 0.112186 0.0386436 0
0.0164928 0.122293 0.0314234 0
-0.0248365 0.156574 -0.00377469 0
0.0452628 0.0863563 0.0200367 0
-0.071339 0.155715 0.00712639 0
-0.0100758 0.125949 0.0297379 0
-0.0720133 0.154518 0.0303984 0
    
```

OpenGL Pipeline

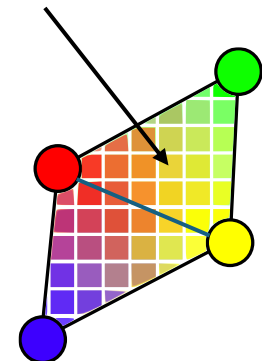
We link together the points
And break the triangles into
Smaller triangles (primitives)



```
MeshVersionFormatted 1
Dimension 3
Vertices
5433
-0.0260146 0.112578 0.0363871 0
-0.0321783 0.174119 -0.00263321 0
-0.080718 0.152855 0.0302446 0
-0.0231294 0.112186 0.0386436 0
0.0164928 0.122293 0.0314234 0
-0.0248365 0.156574 -0.00377469 0
0.0452628 0.0863563 0.0200367 0
-0.071339 0.155715 0.00712639 0
-0.0100758 0.125949 0.0297379 0
-0.0720133 0.154518 0.0303984 0
```

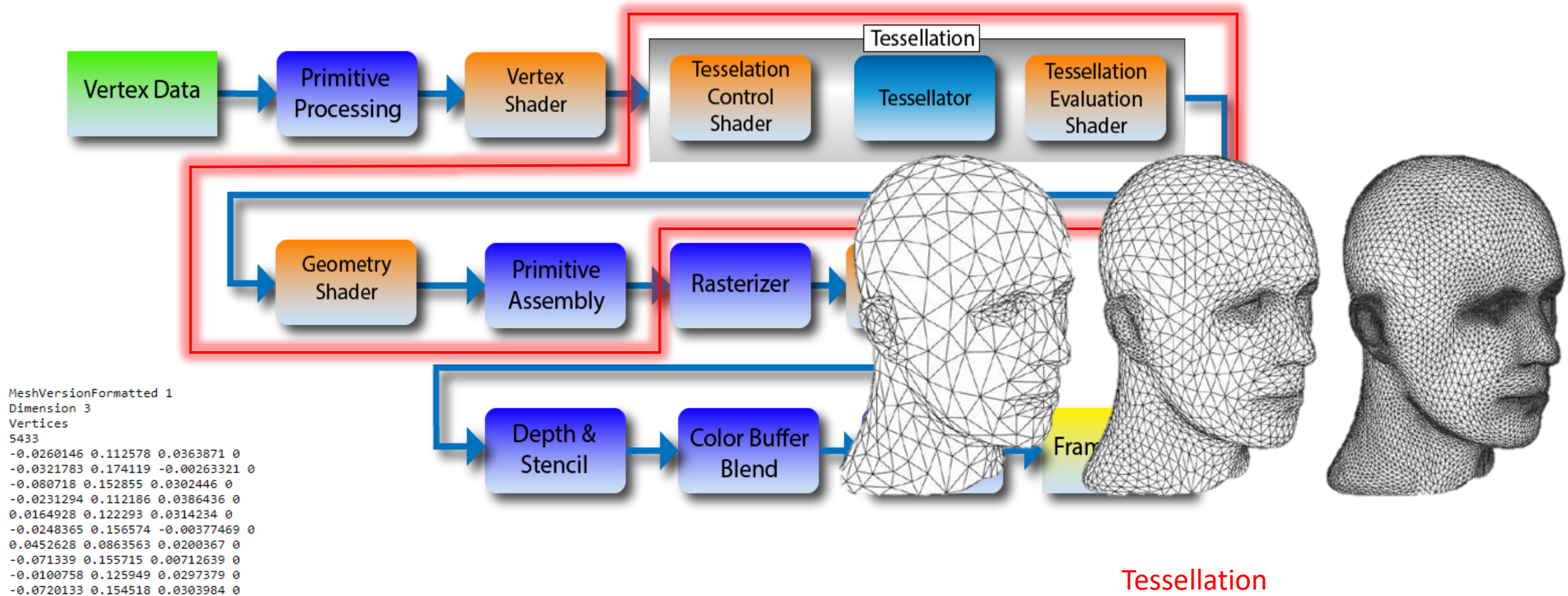
3 vertices = one triangle → Divided in many fragments (patches)

Fragment

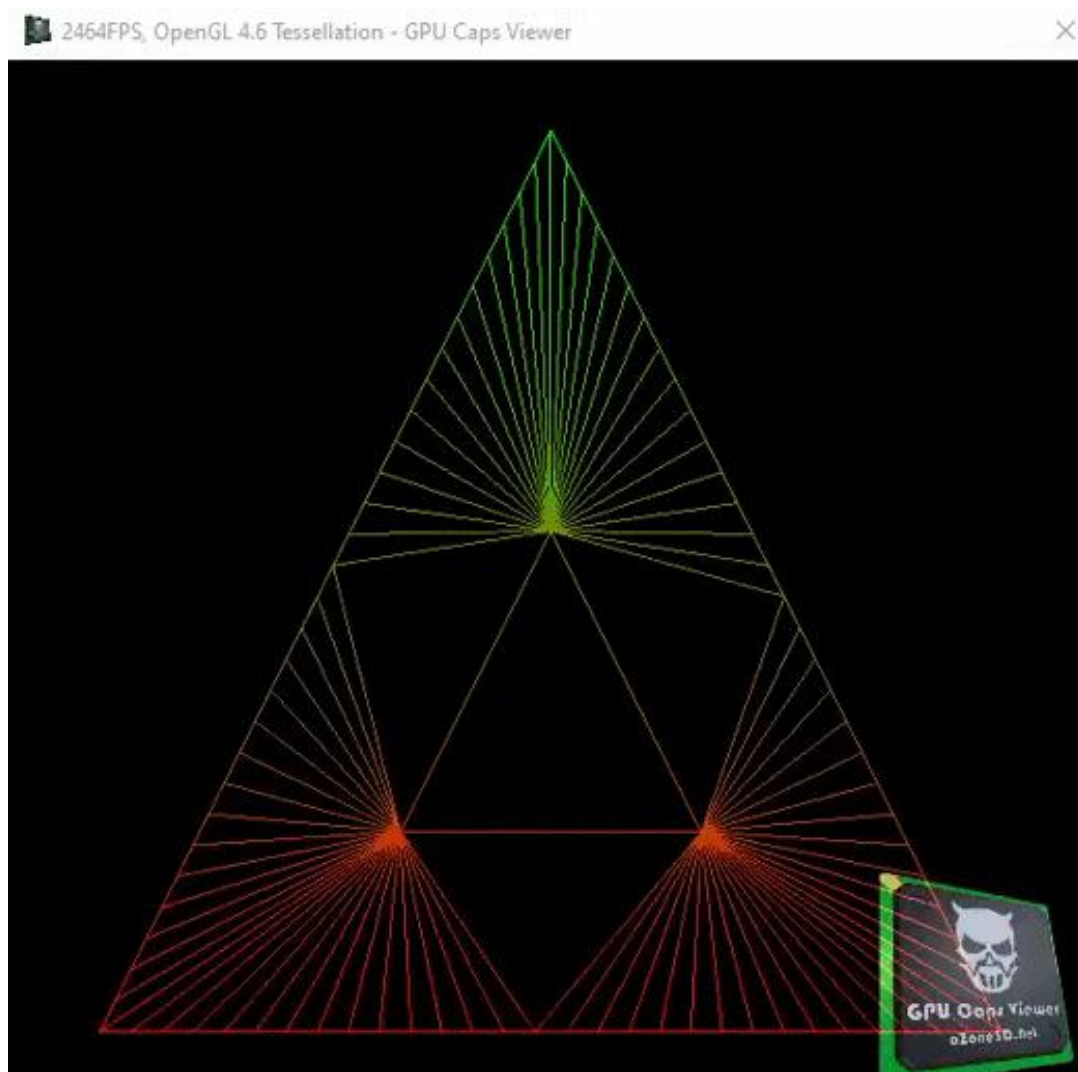


OpenGL Pipeline

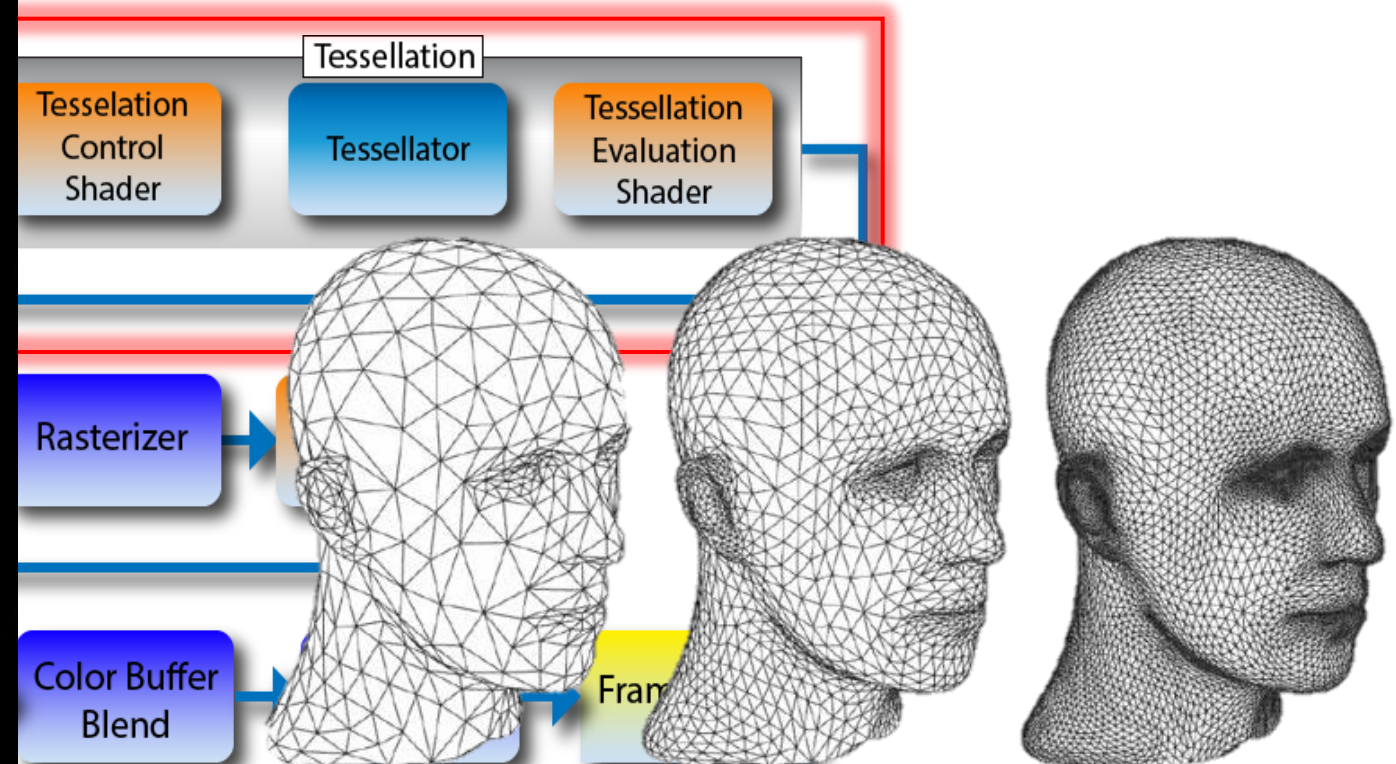
We link together the points
And break the triangles into
Smaller triangles (primitives)



OpenGL Pipeline



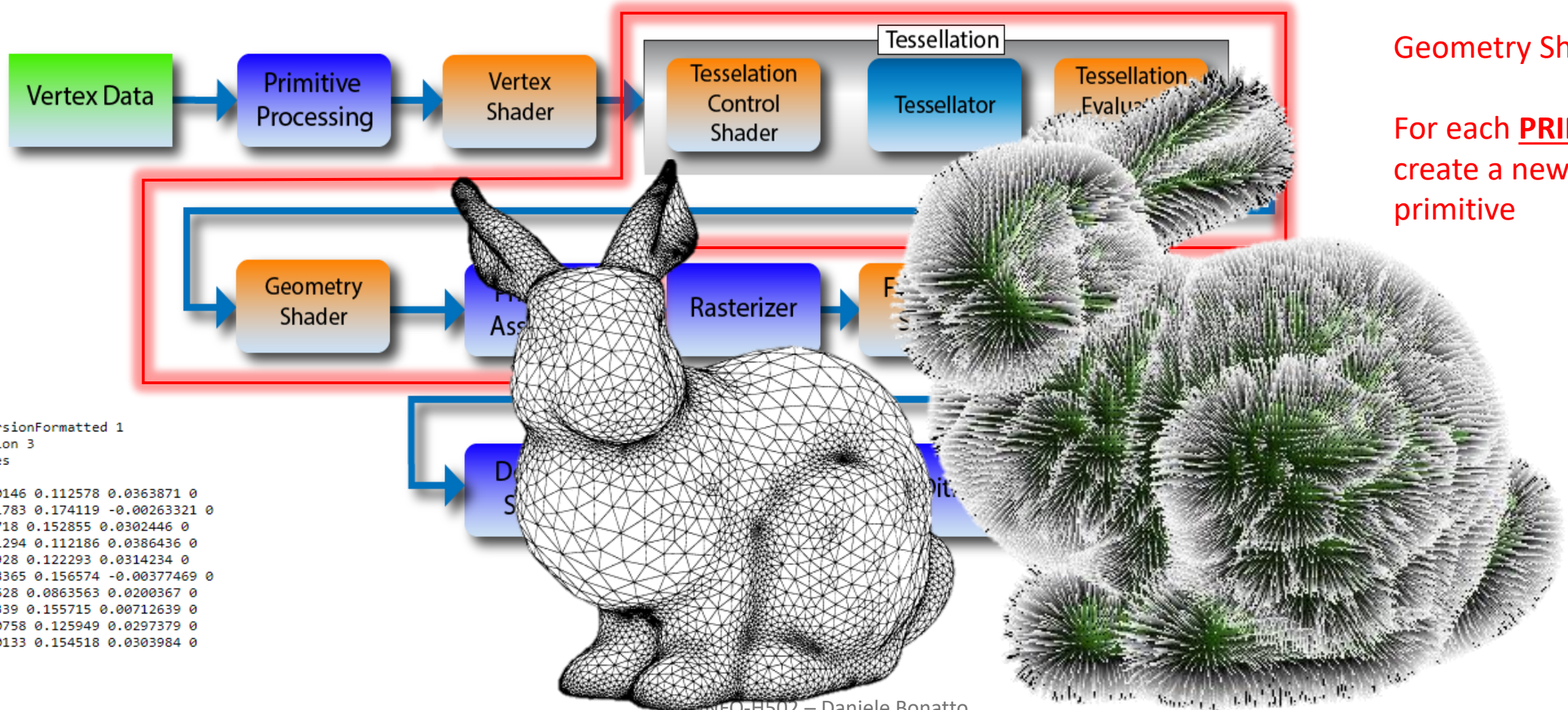
We link together the points
And break the triangles into
Smaller triangles (primitives)



Tessellation

OpenGL Pipeline

We link together the points
And break the triangles into
Smaller triangles (primitives)



Geometry Shader:

For each **PRIMITIVE**:
create a new
primitive

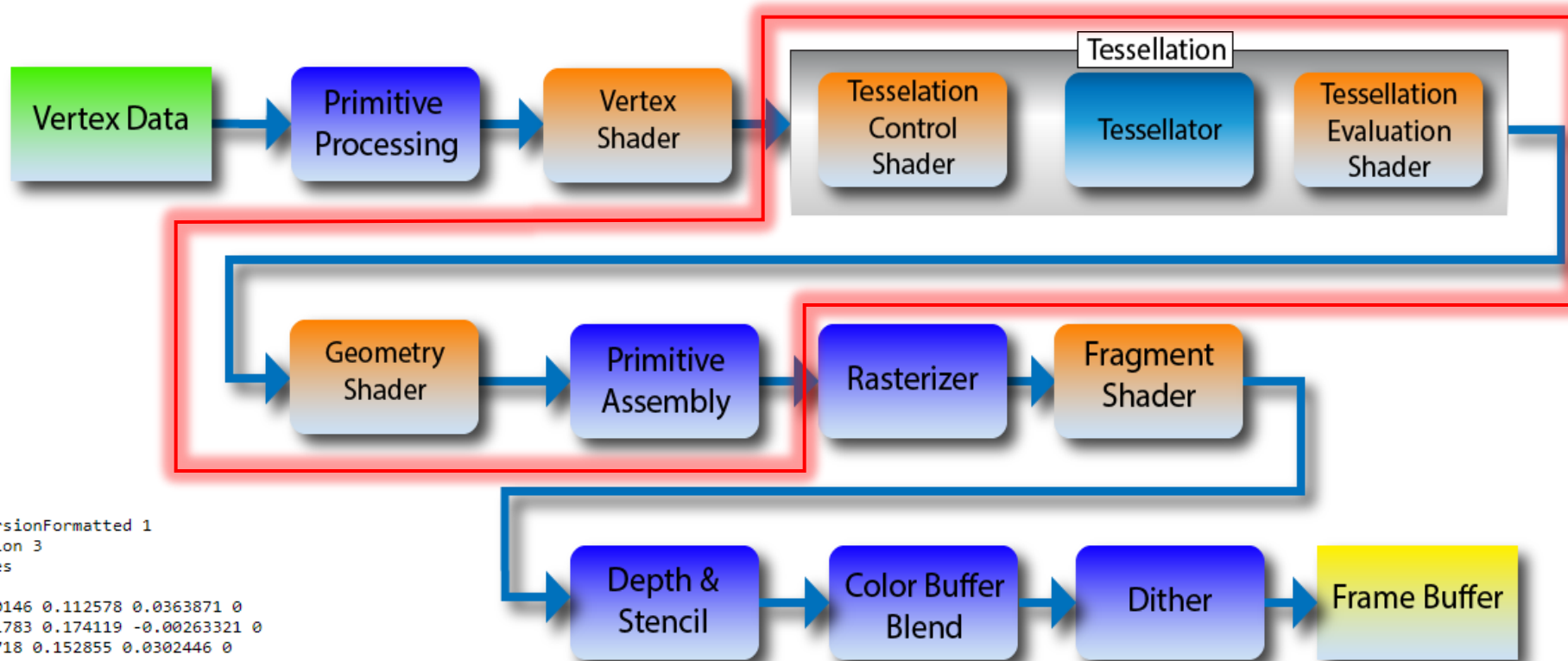
```
MeshVersionFormatted 1
Dimension 3
Vertices
5433
-0.0260146 0.112578 0.0363871 0
-0.0321783 0.174119 -0.00263321 0
-0.080718 0.152855 0.0302446 0
-0.0231294 0.112186 0.0386436 0
0.0164928 0.122293 0.0314234 0
-0.0248365 0.156574 -0.00377469 0
0.0452628 0.0863563 0.0200367 0
-0.071339 0.155715 0.00712639 0
-0.0100758 0.125949 0.0297379 0
-0.0720133 0.154518 0.0303984 0
```

OpenGL Pipeline

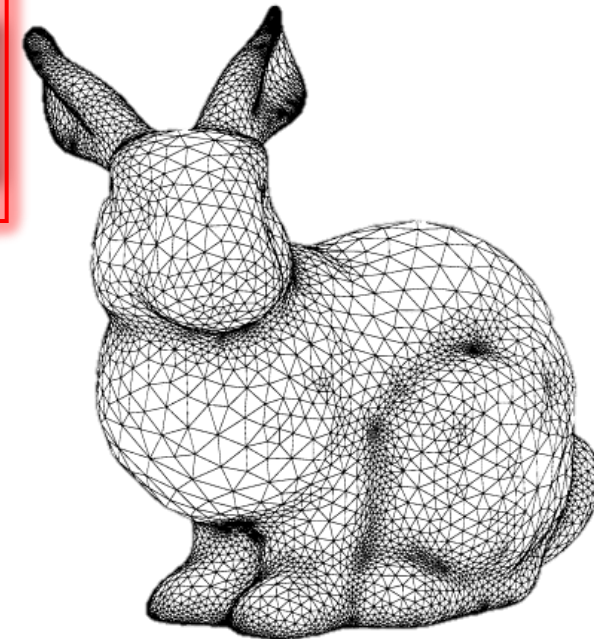
We link together the points
And break the triangles into
Smaller triangles (primitives)

Primitive Assembly

Link all the points!



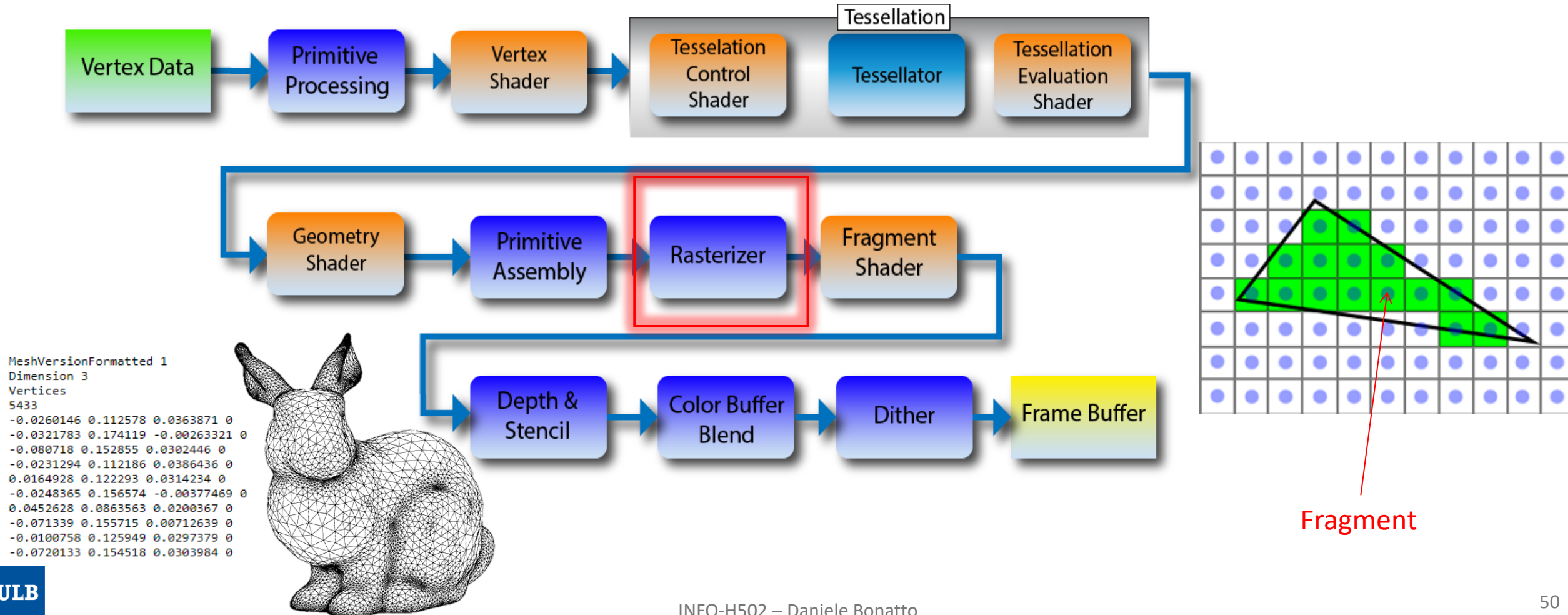
```
MeshVersionFormatted 1
Dimension 3
Vertices
5433
-0.0260146 0.112578 0.0363871 0
-0.0321783 0.174119 -0.00263321 0
-0.080718 0.152855 0.0302446 0
-0.0231294 0.112186 0.0386436 0
0.0164928 0.122293 0.0314234 0
-0.0248365 0.156574 -0.00377469 0
0.0452628 0.0863563 0.0200367 0
-0.071339 0.155715 0.00712639 0
-0.0100758 0.125949 0.0297379 0
-0.0720133 0.154518 0.0303984 0
```



OpenGL Pipeline

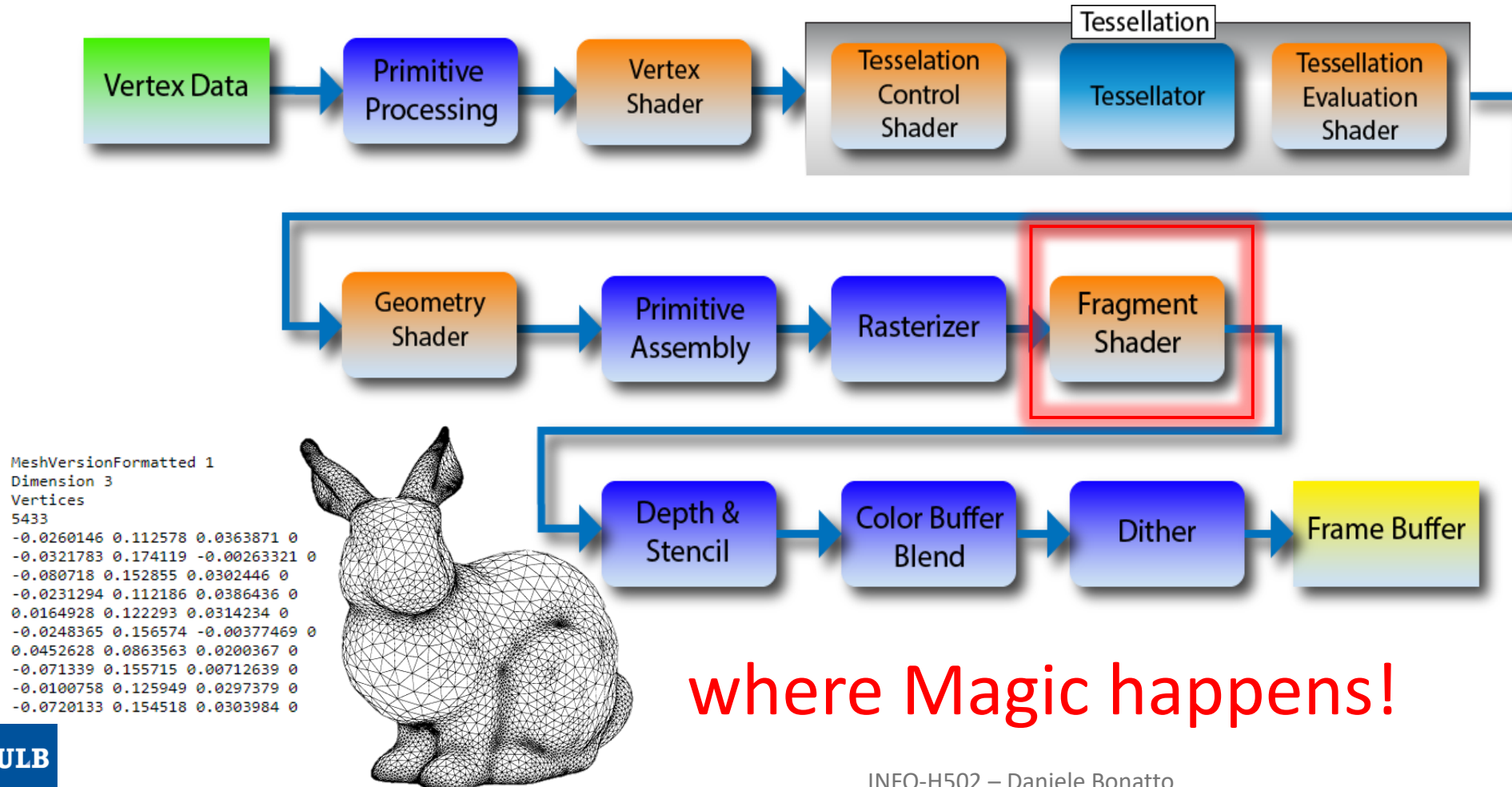
triangles (primitives)
Are broken down into
Fragments

Computer Screen
=
array of pixels

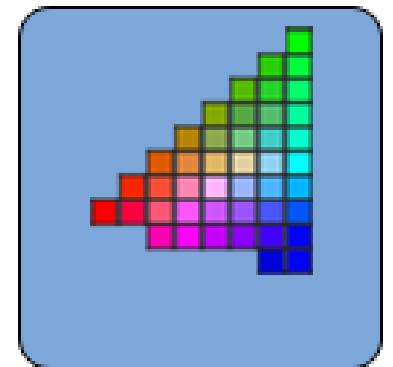
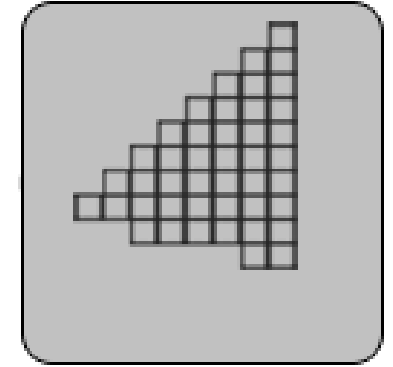


OpenGL Pipeline

EACH fragment is colored
Independently (GPU threads)

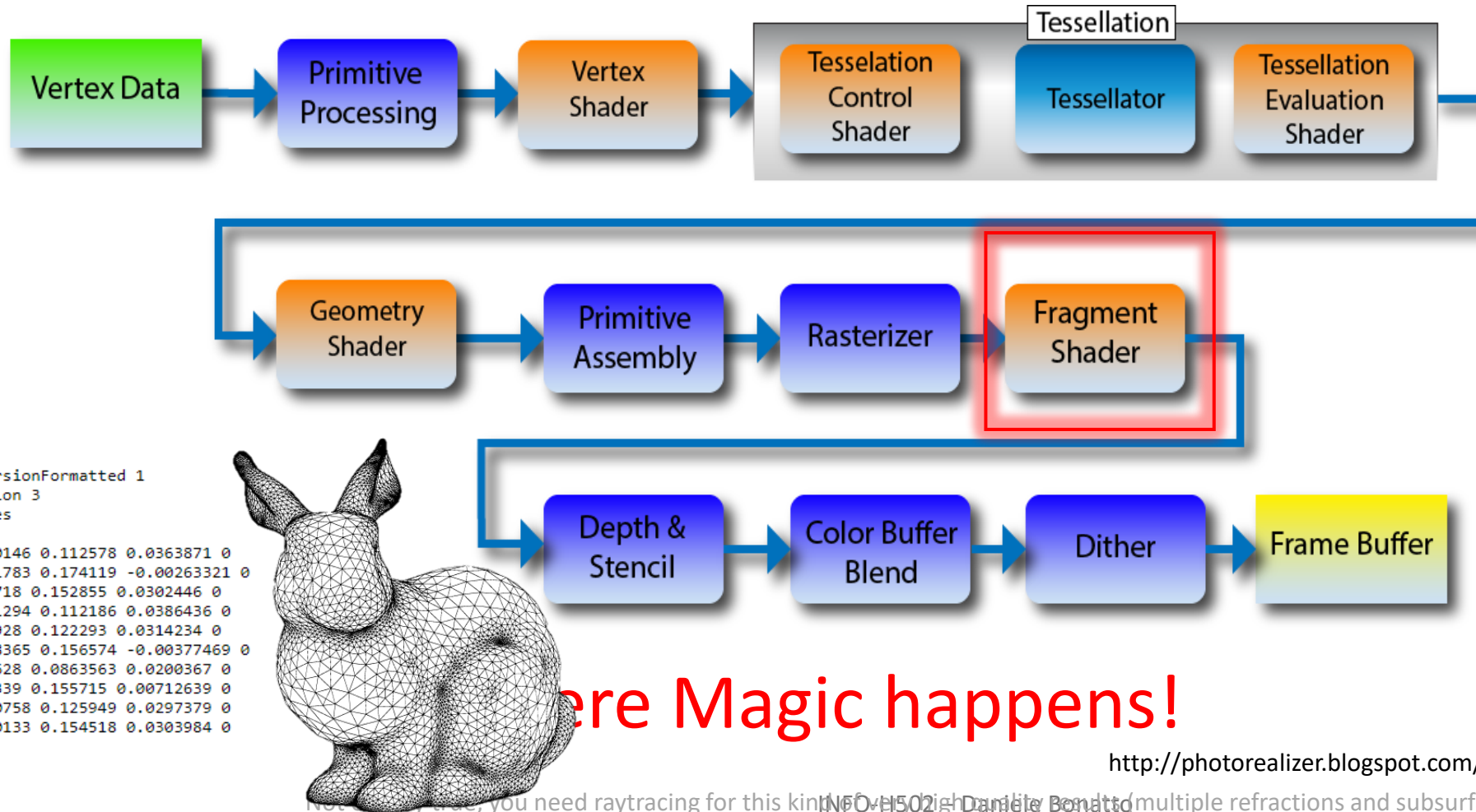


where Magic happens!



OpenGL Pipeline

EACH fragment is colored
Independently (GPU threads)
This is where the magic happens

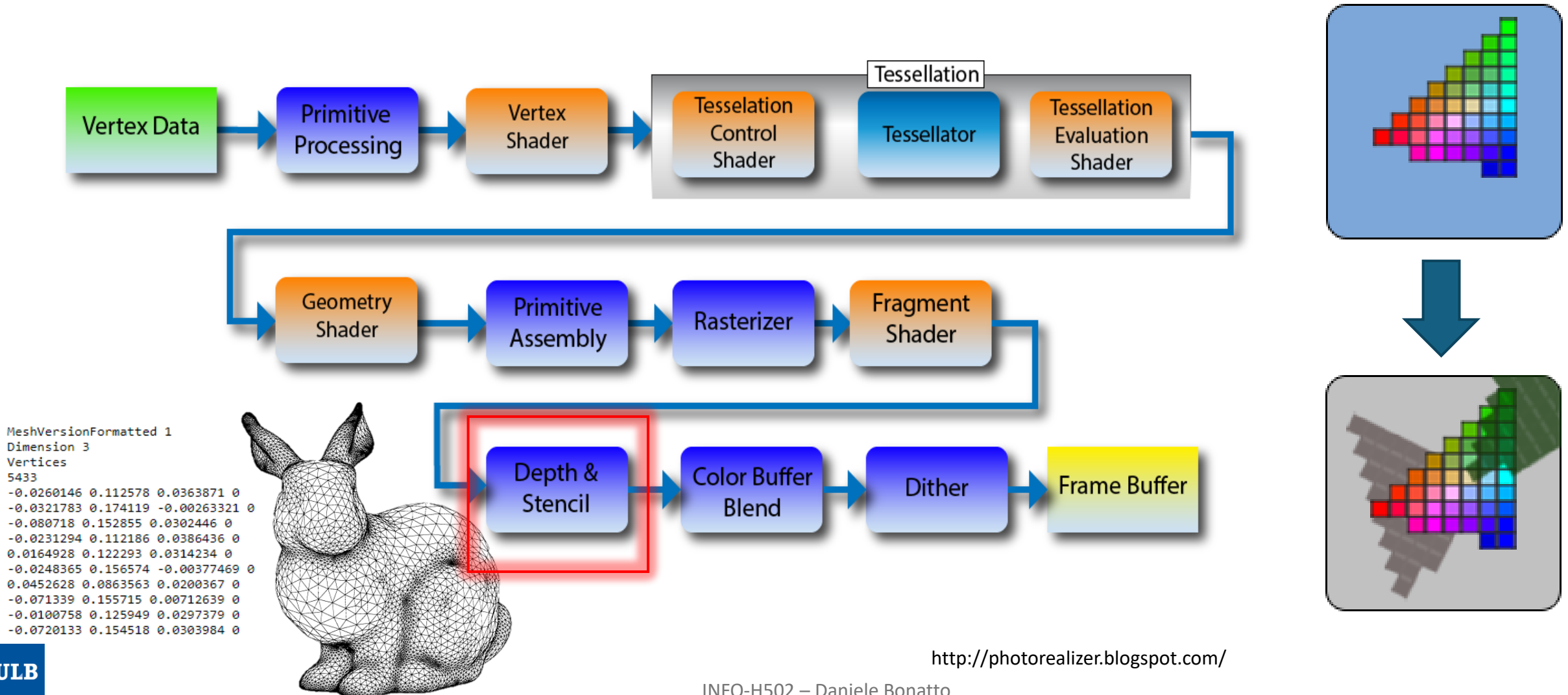


<http://photorealizer.blogspot.com/>

For this kind of rendering, you need raytracing for this kind of rendering (multiple refractions and subsurface scattering), but we can go very close!

OpenGL Pipeline

If there were overlapping triangles
We select the one to show

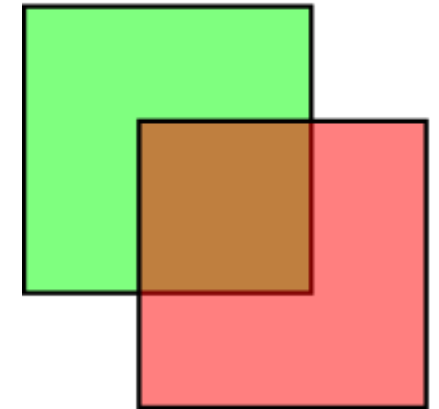


OpenGL Pipeline

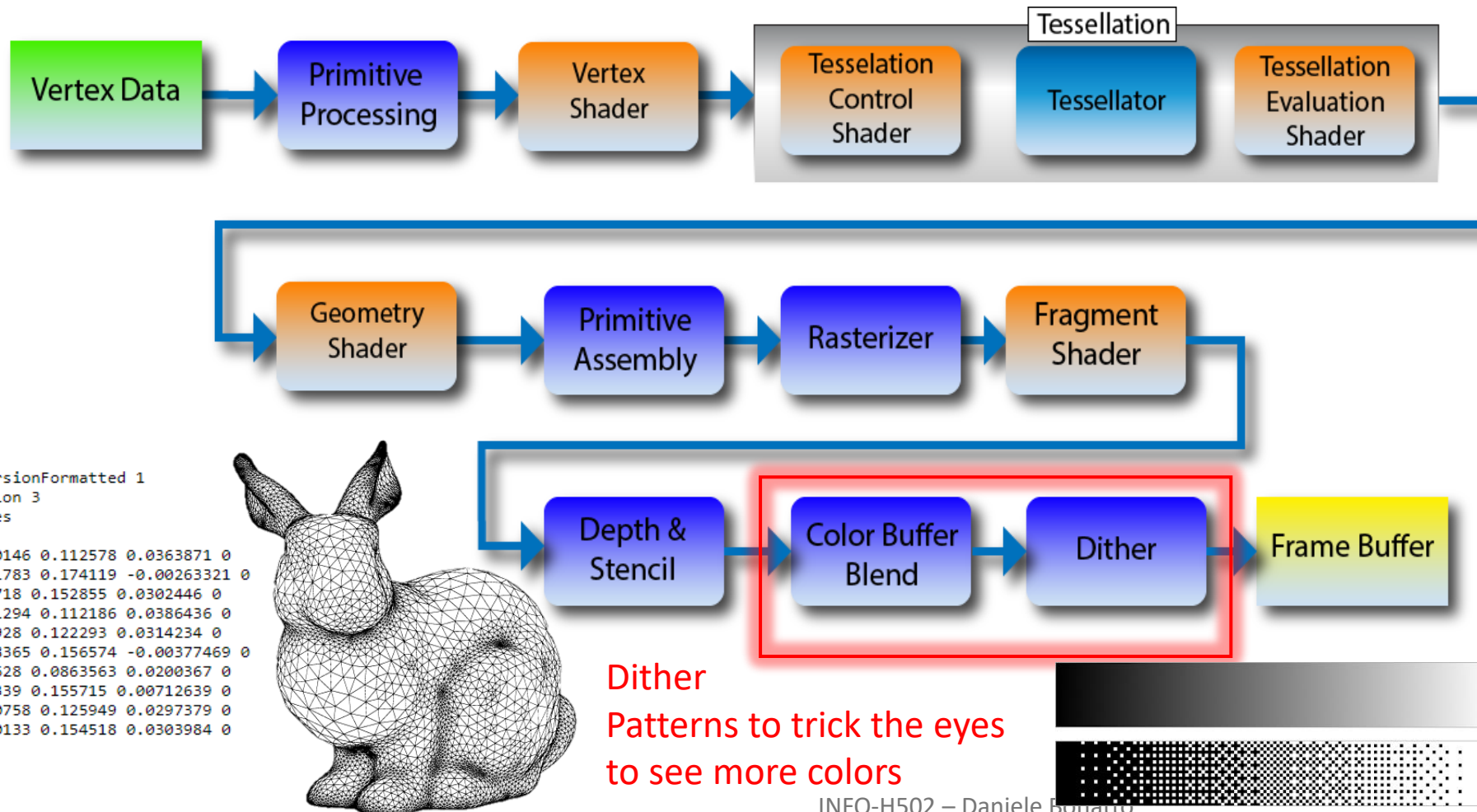
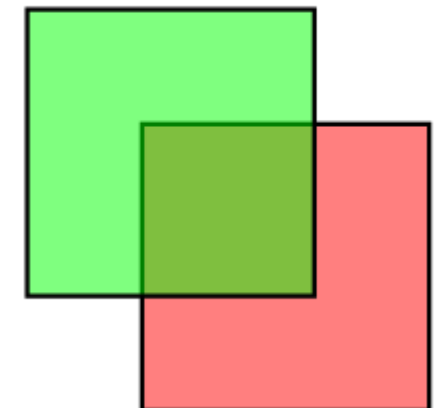
Small post processing
Eg:
- Normalization of the colors
- Alpha channel blending

Color = R8G8B8A8

Red on top



Green on top

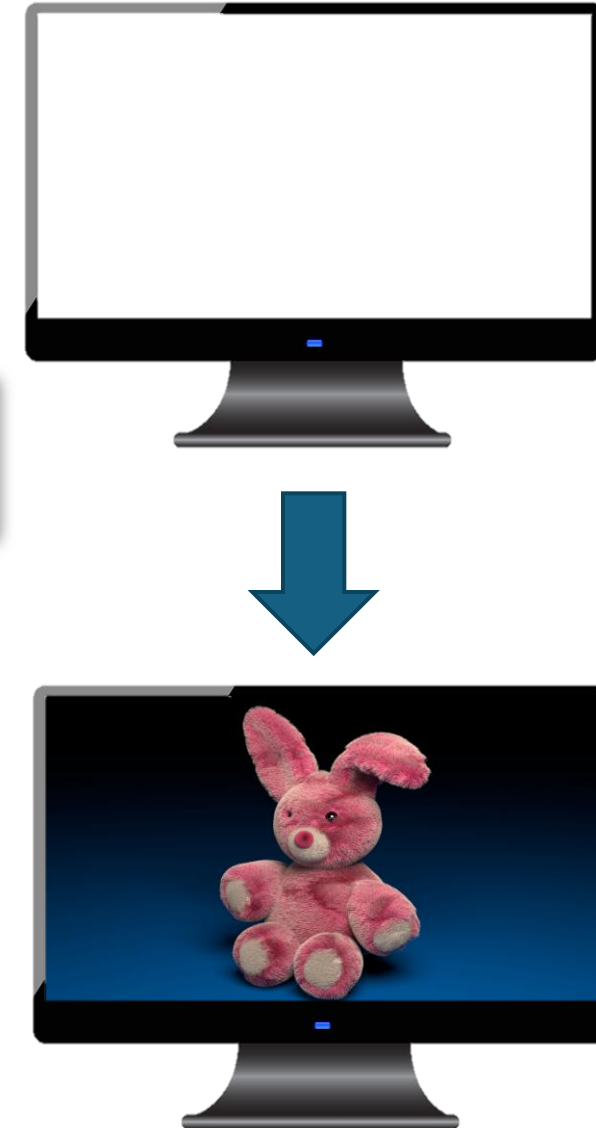
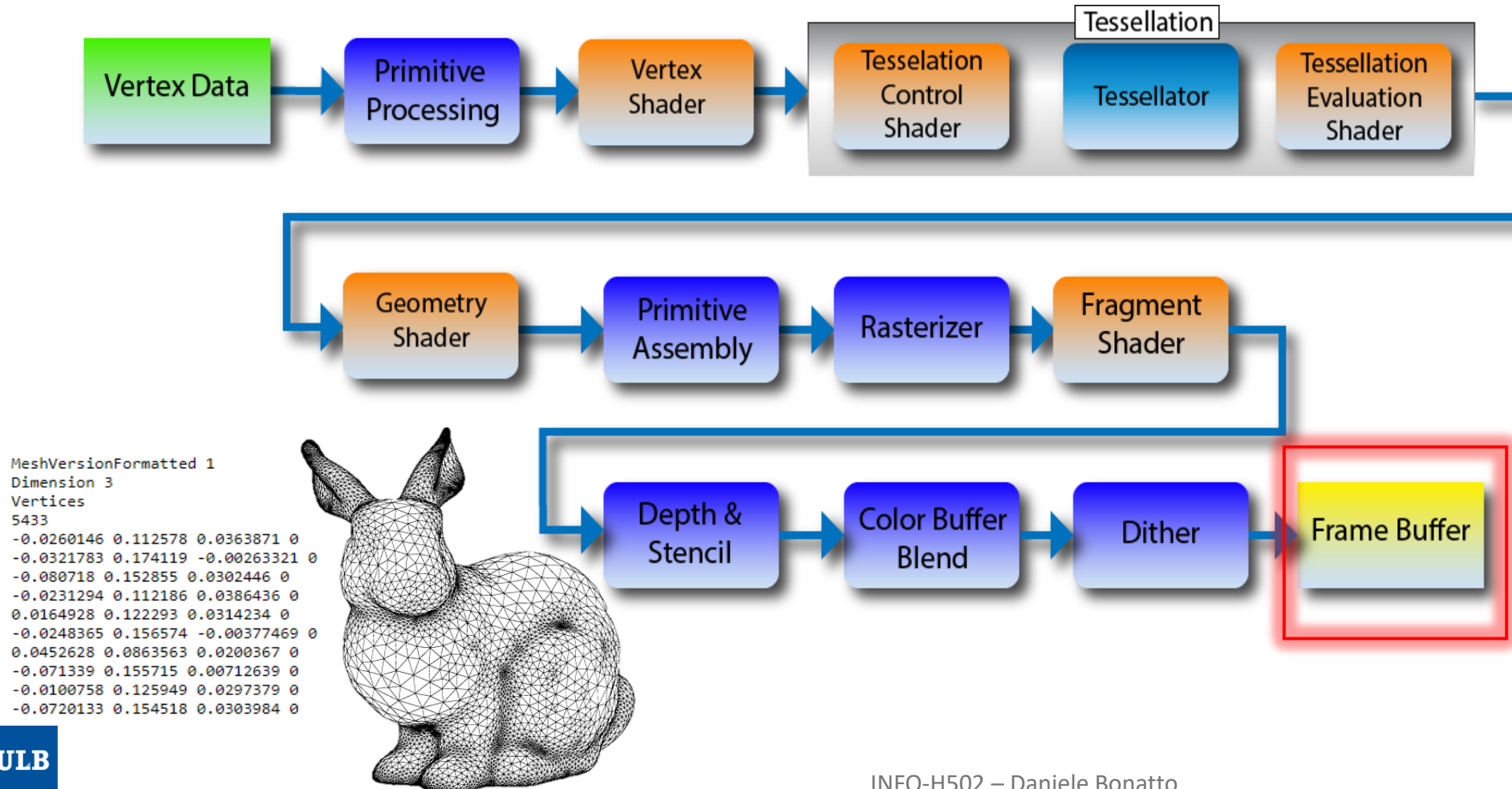


```
MeshVersionFormatted 1
Dimension 3
Vertices
5433
-0.0260146 0.112578 0.0363871 0
-0.0321783 0.174119 -0.00263321 0
-0.080718 0.152855 0.0302446 0
-0.0231294 0.112186 0.0386436 0
0.0164928 0.122293 0.0314234 0
-0.0248365 0.156574 -0.00377469 0
0.0452628 0.0863563 0.0200367 0
-0.071339 0.155715 0.00712639 0
-0.0100758 0.125949 0.0297379 0
-0.0720133 0.154518 0.0303984 0
```

Dither
Patterns to trick the eyes
to see more colors

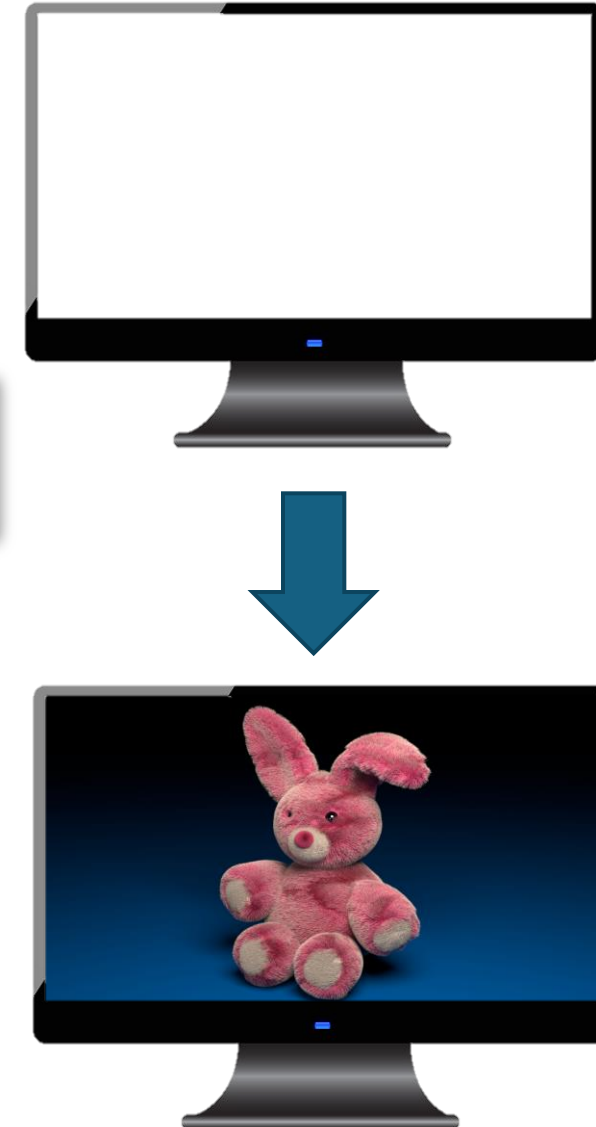
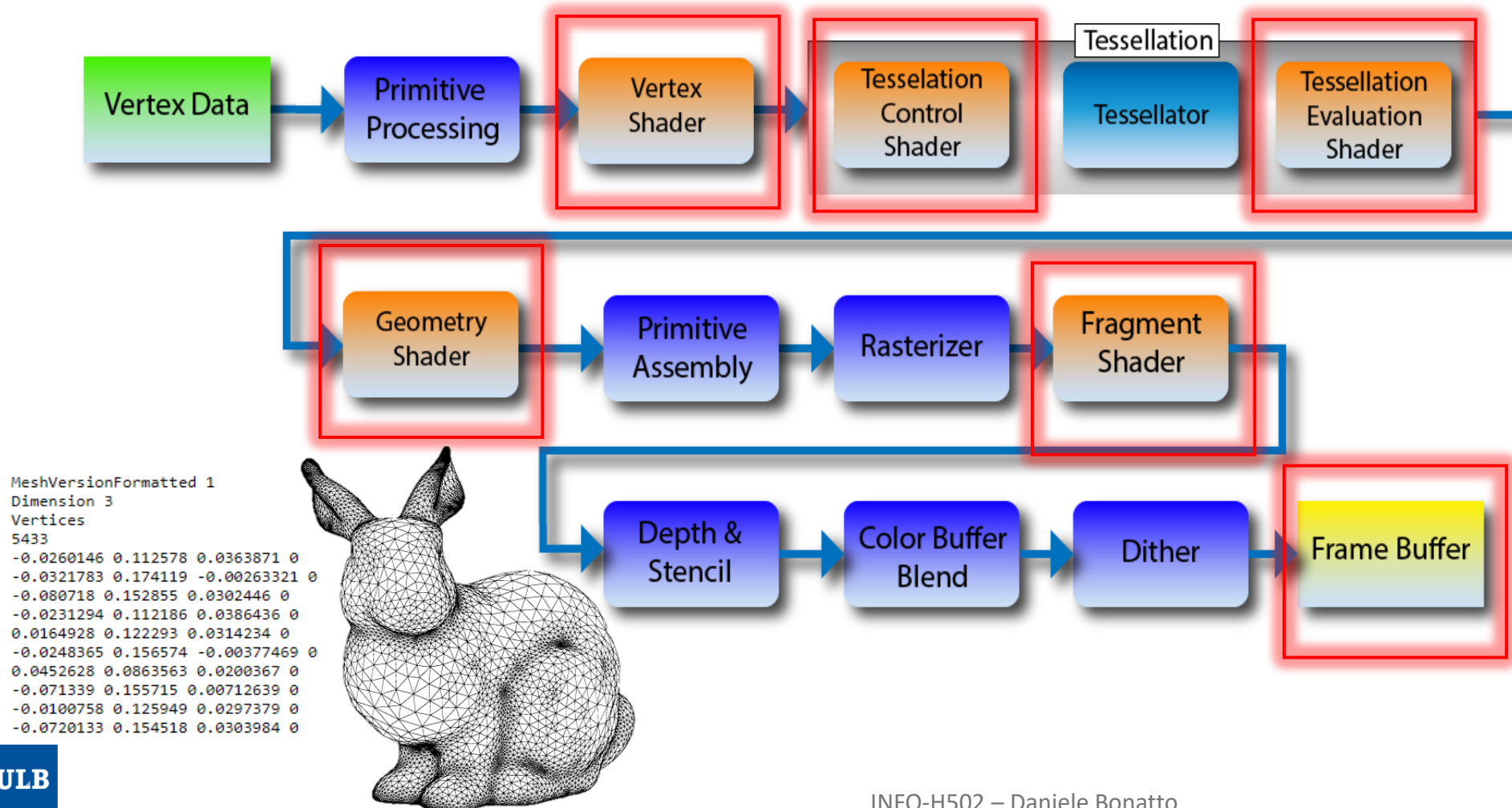
OpenGL Pipeline

A special “Buffer” (matrix)
Which will be displayed on screen
(or used for effects!)



OpenGL Pipeline

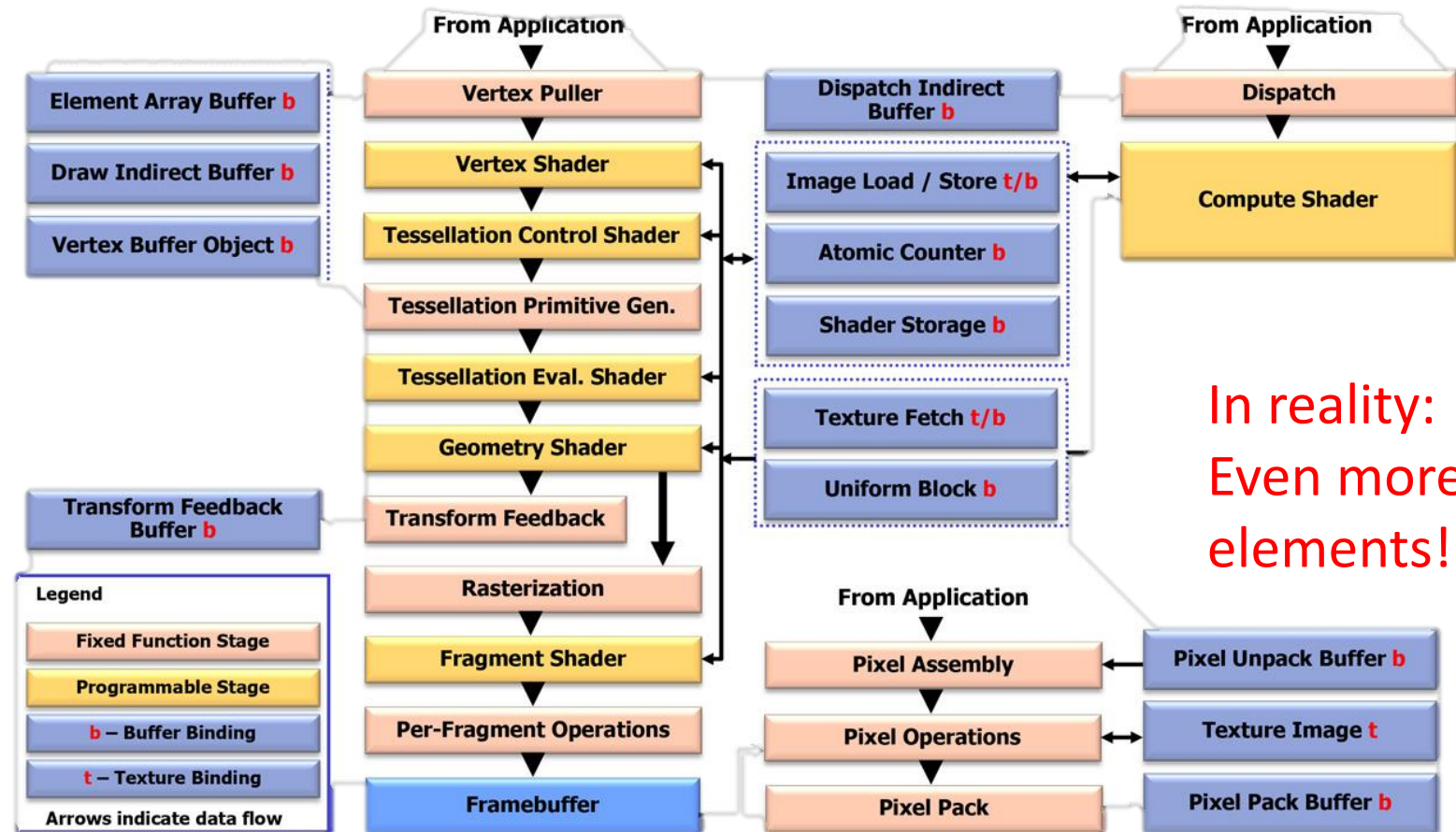
Orange stages are programmable
- They depends on the framework
- WebGL, OpenGL2, OpenGL 3, OpenGL ES



There is actually more in the OpenGL pipeline

- In computer graphics we are also interested in iterative algorithms for
 - Clothes simulations
 - Liquid simulations
 - Particle systems

- We use the
 - Transform feedback



Transform Feedback – Compute shader

- Using shaders for computing
- Modern algorithms do arithmetics on the framebuffer color components instead of doing rendering!
- Compute shader
 - loop that can read from the fragment shader and send the data again to the vertex shader without going through the CPU



NVIDIA® Real Time GPU Based 3D Fluid and Particle Simulation and Rendering Using CUDA™

Rigid Fluid: Animating the Interplay Between Rigid Bodies and Fluid

Mark Carlson
Peter J. Mucha
Greg Turk

Georgia Institute of Technology

Sound FX by Andrew Lackey, M.P.S.E.

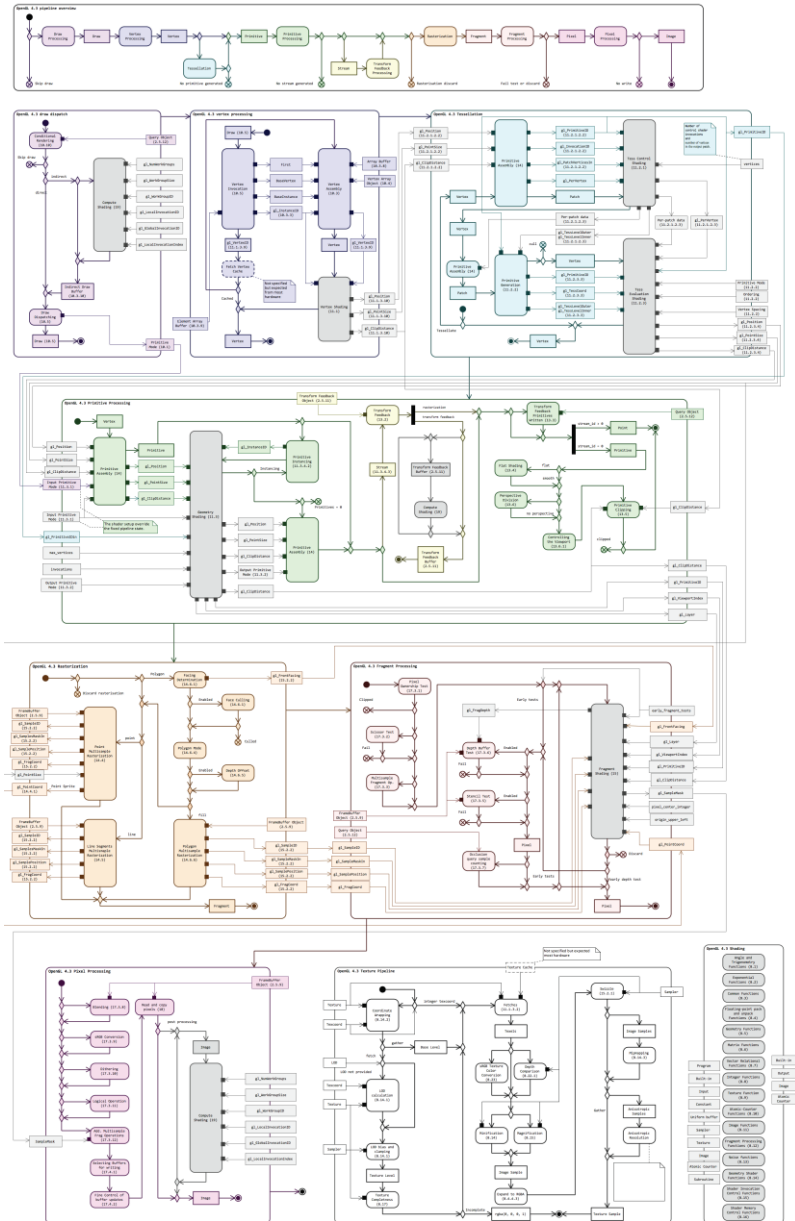
Interactive Cloth Simulation
<https://www.youtube.com/watch?v=KBfxnaylIOY>

Particle Systems
Real Time 3D Fluid and Particle Simulation and Rendering
<https://www.youtube.com/watch?v=RuZQpWo9Qhs>

Rigid Body & Fluid Dynamics
<https://www.youtube.com/watch?v=aLwYWW2N16w>

The complete OpenGL Pipeline

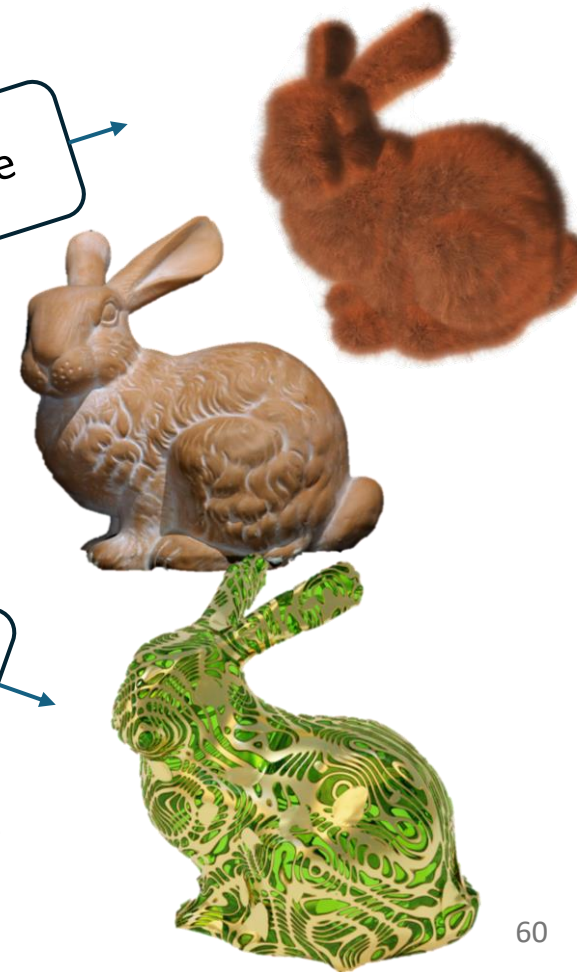
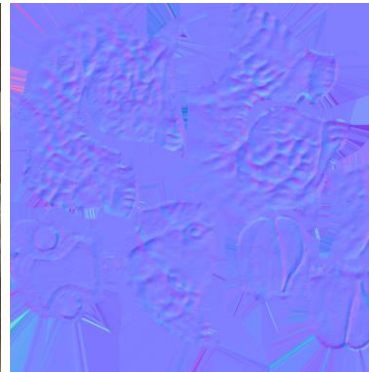
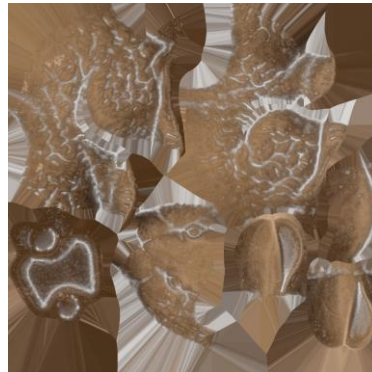
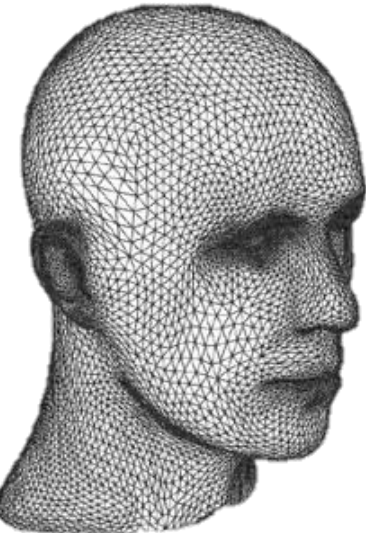
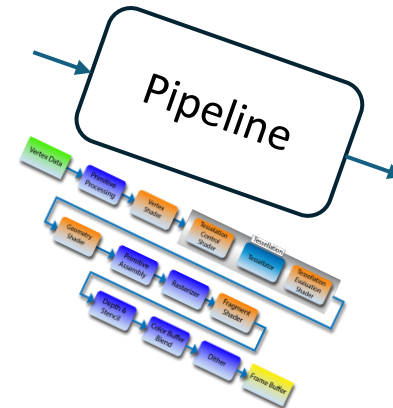
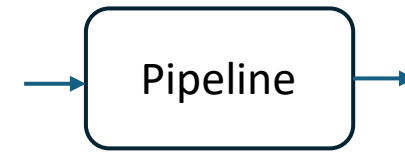
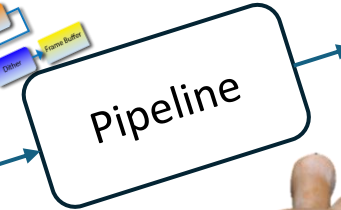
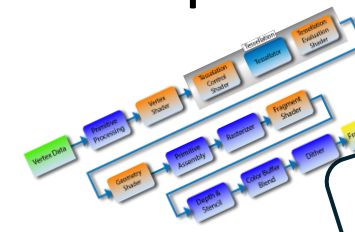
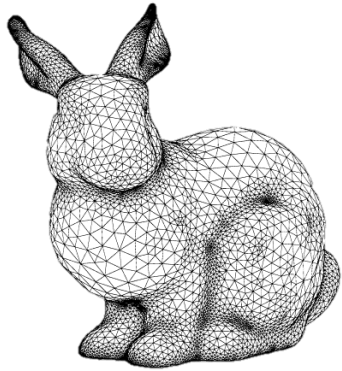
- We will not cover it in those lectures
- We focus on main parts:
 - Vertex shader
 - Geometry shader
 - Fragment shader



<https://www.g-truc.net/doc/OpenGL%204.3%20Pipeline%20Map.pdf>

All together in a software

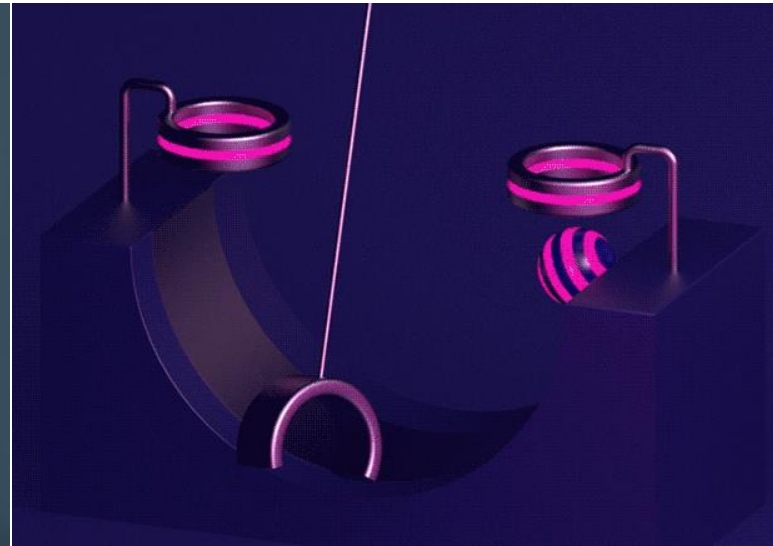
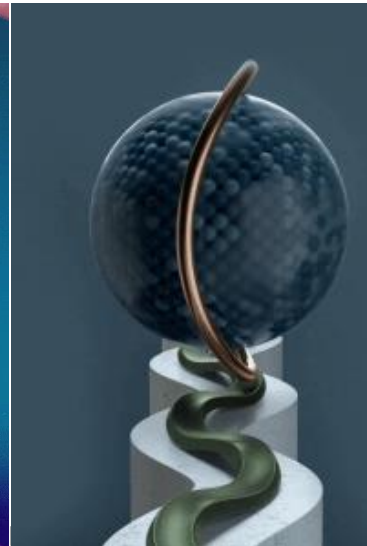
- We have objects, textures, colors, normals, a camera, lights...
- We will show a frame on the screen from the point of view of the camera.
- For each object, we apply different pipelines composed of different shaders.



The Rendering Loop

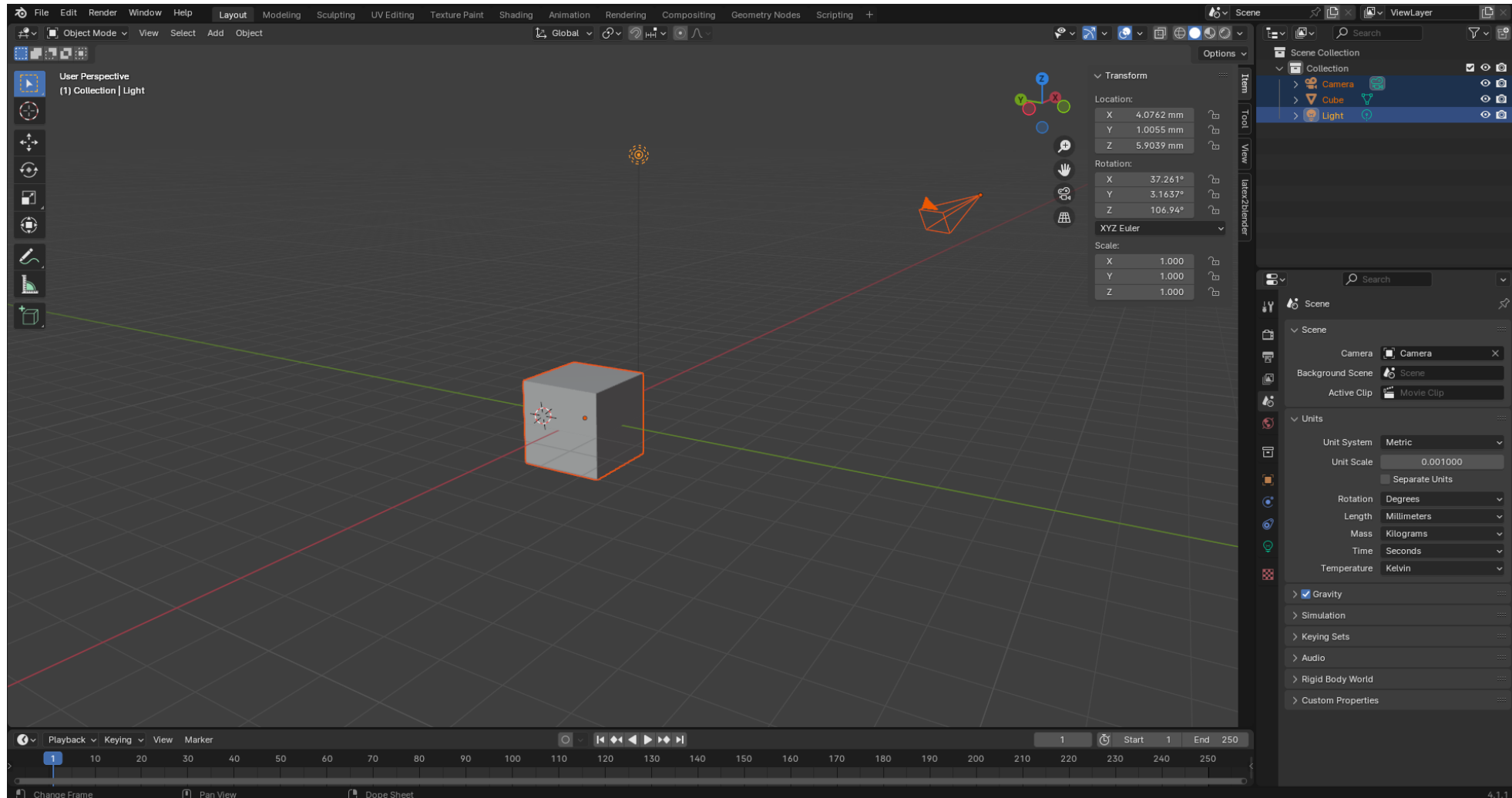
- Usually, 3D is animated in time.
- We use an infinite loop that x frames every seconds (Frames per Second FPS)
- In this loop, we update the position of objects, textures, deformations, etc.

```
var deltaTime = 0;  
function animate (time) {  
  
    //DRAW EVERYTHING  
  
    deltaTime += 0.005;  
  
    fps(time); // show the FPS on screen TO BE DONE  
    window.requestAnimationFrame(animate); // While(True) loop!  
}  
  
animate(0);
```



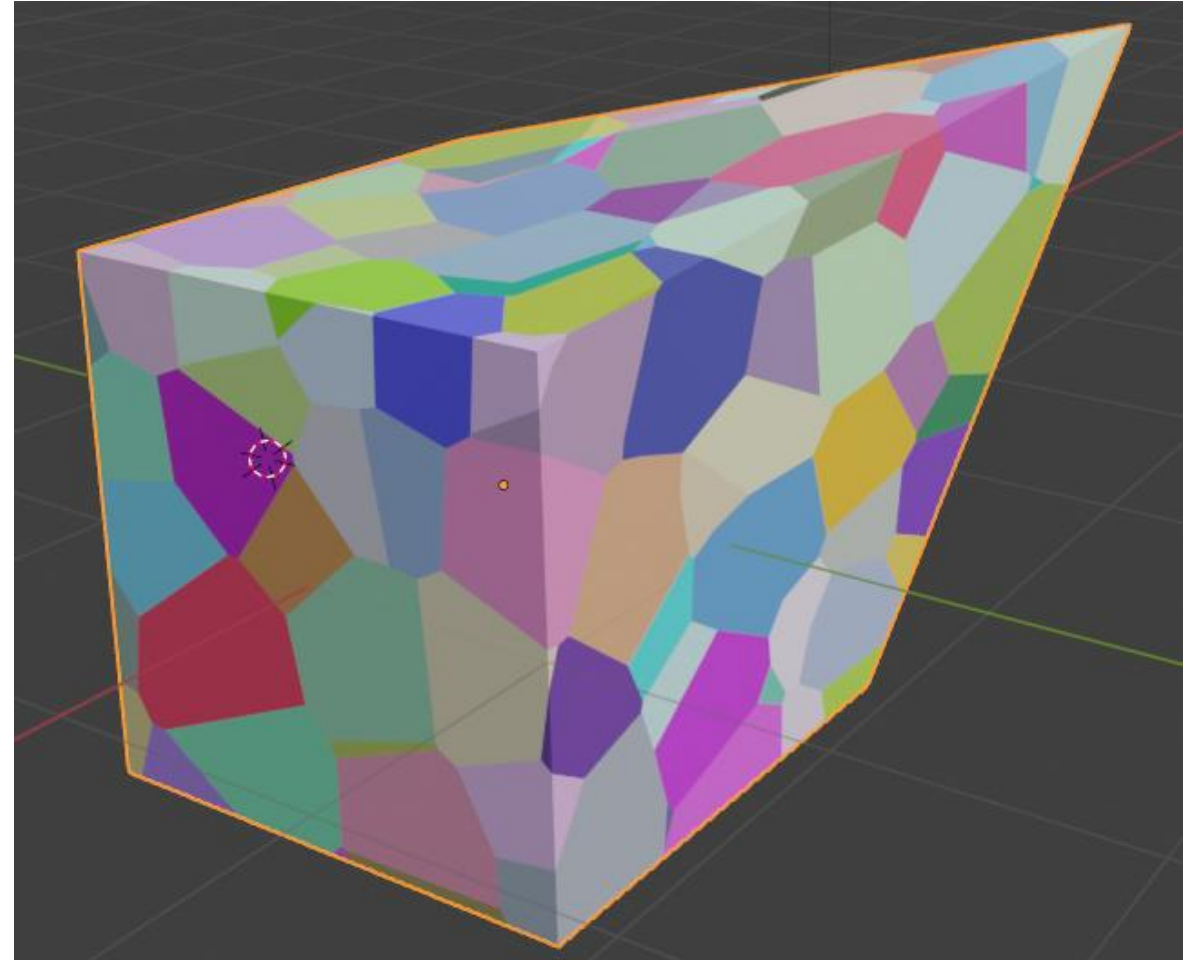
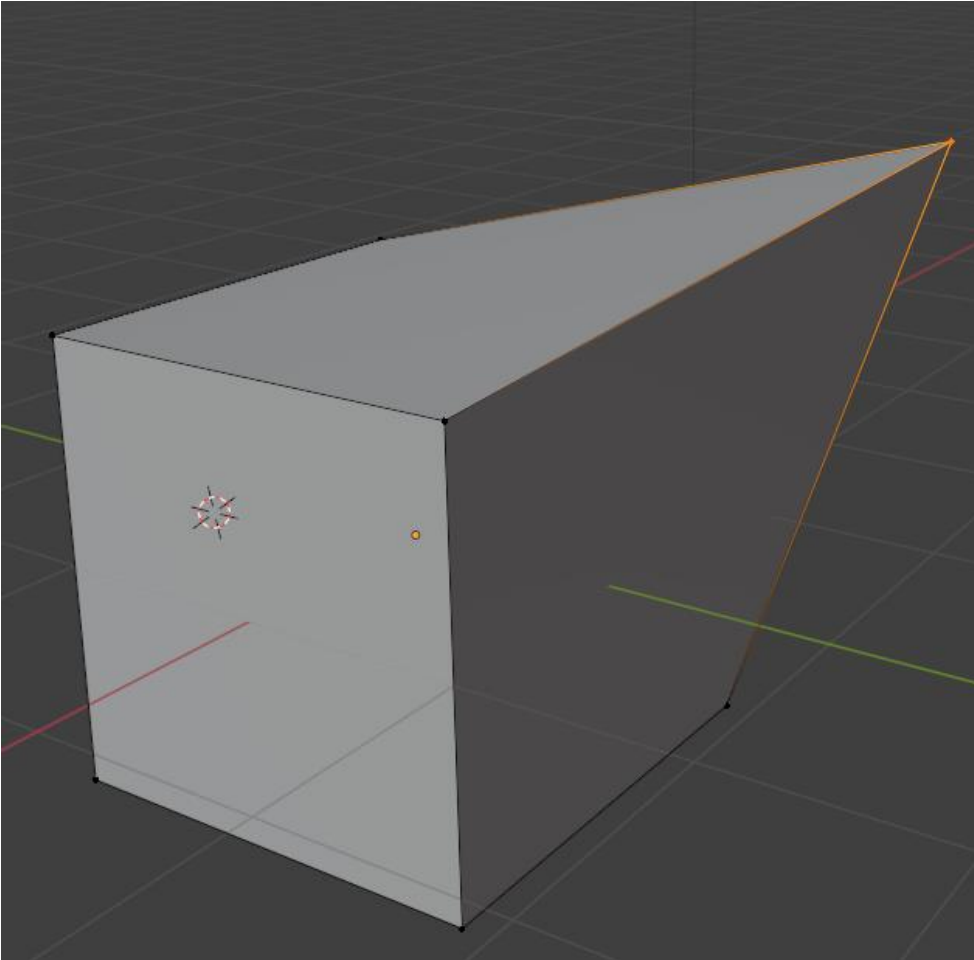
Blender

- To make 3D objects we use the Blender software
- <https://www.blender.org/>



Blender

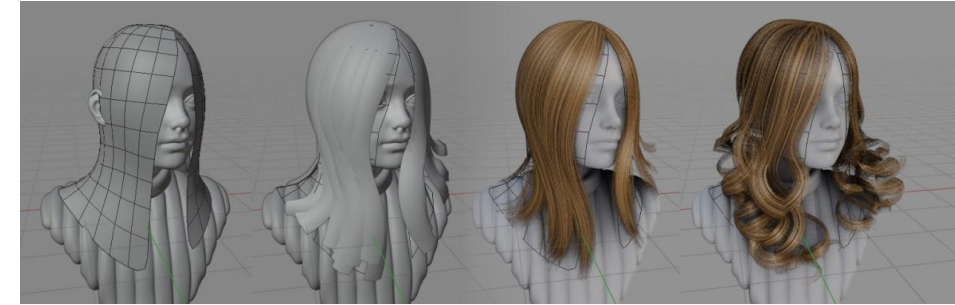
- Blender allows to move vertices, edges and apply textures



Blender

- It takes times to master Blender, play with it when you can
- It will not be required to learn it for the exercises
- But it will be extremely useful for the project

Hairs simulation



Modeling

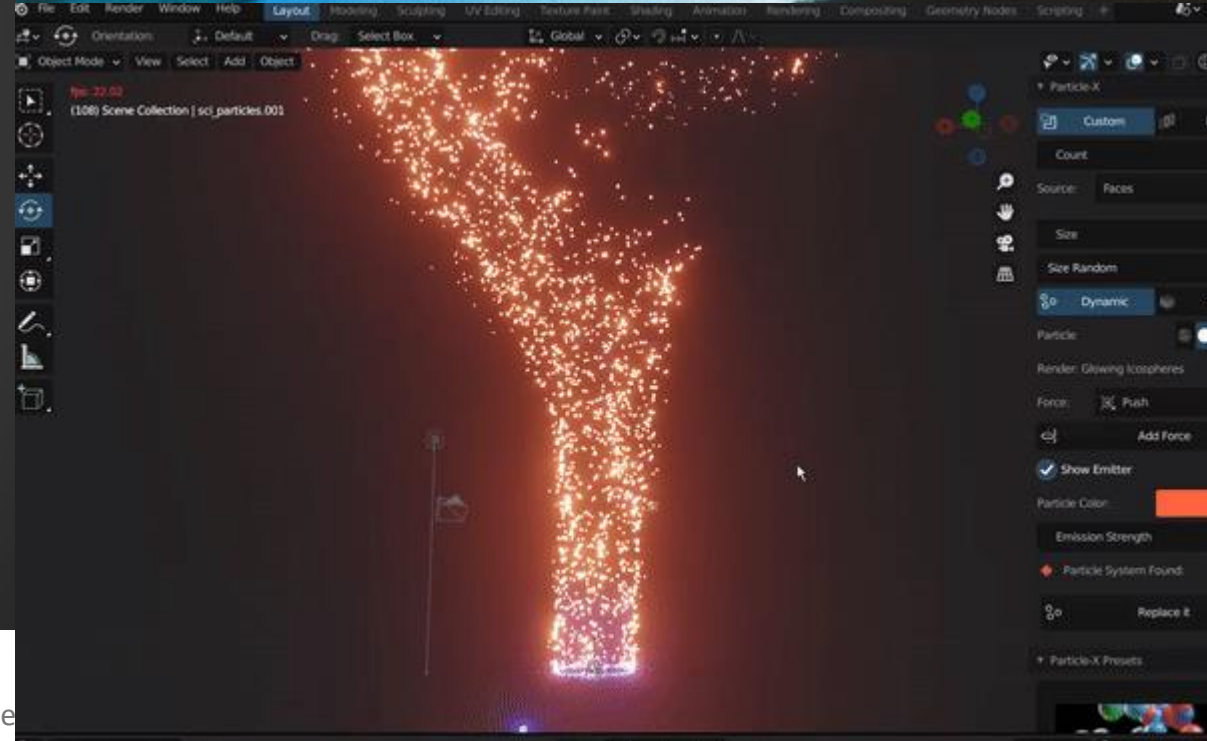


Animations

Blender

- And way more

Physical simulations



Sculpting

Course evaluation

- One project:
 - Data transfers
 - Linear algebra
 - OpenGL pipeline
 - Make it *beautiful*
- One oral exam
 - Questions on **any** parts of the slides
 - Selected paper presentation

Virtual Reality Project

Roller coaster

INFO-H-502



ECOLE
POLYTECHNIQUE
DE BRUXELLES



Next time

- Next lesson is more mathematical
- But mostly we revisit lectures that you had in the past
 - In the computer graphics perspective
- We will build ALL the mathematics needed from nothing to manipulate our 3D objects !
- Don't miss it !

Temptative Table of Content

- 1. Introduction to computer graphics
- 2. Mathematical foundations 1
- 3. Mathematical foundations 2
- 4. Perspective matrix transform
- 5. Light equations and Raytracing
- 6. Textures, bump mapping, depth and alpha
- 7. Encoding geometry and NURBS
- 8. Physically based animations aka Simulations (fluids mechanics)
- 9. Advanced material and VR